



DOSW COMPANY



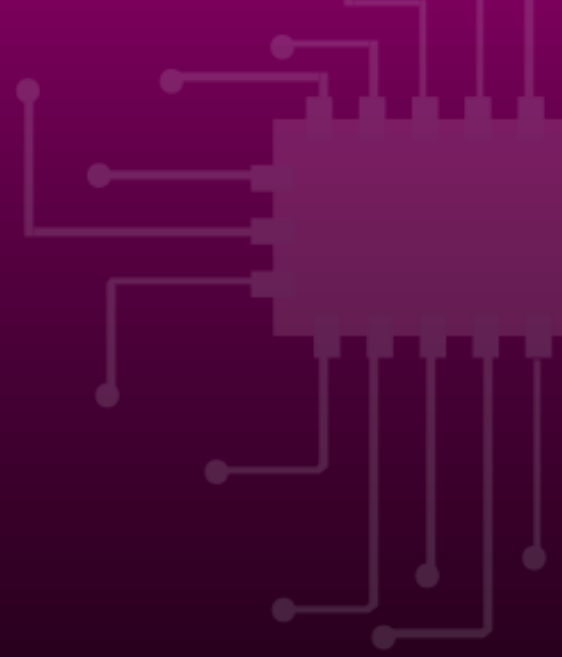
ESCUELA COLOMBIANA DE INGENIERIA

SEMANA 3

GRUPO 1



ESCUELA COLOMBIANA DE INGENIERIA



AGENDA

- PATRONES DE DISEÑO CREACIONALES
- PATRONES DE DISEÑO ESTRUCTURALES
- PATRONES DE DISEÑO COMPORTAMIENTO



DOSW COMPANY



PATRONES DE DISEÑO

GANG OF 4

No memoricen patrones: aprendan a reconocer el problema de diseño que cada patrón resuelve.

MODULO DE DIAGRAMACION



DIAGRAMACION # 1 - DIAGRAMAS DE CLASES

El diagrama de clases es un diagrama estructural, parte de la familia que modela las relaciones estáticas de un sistema en estado estacionario. Es una herramienta vital para la programación orientada a objetos (POO).

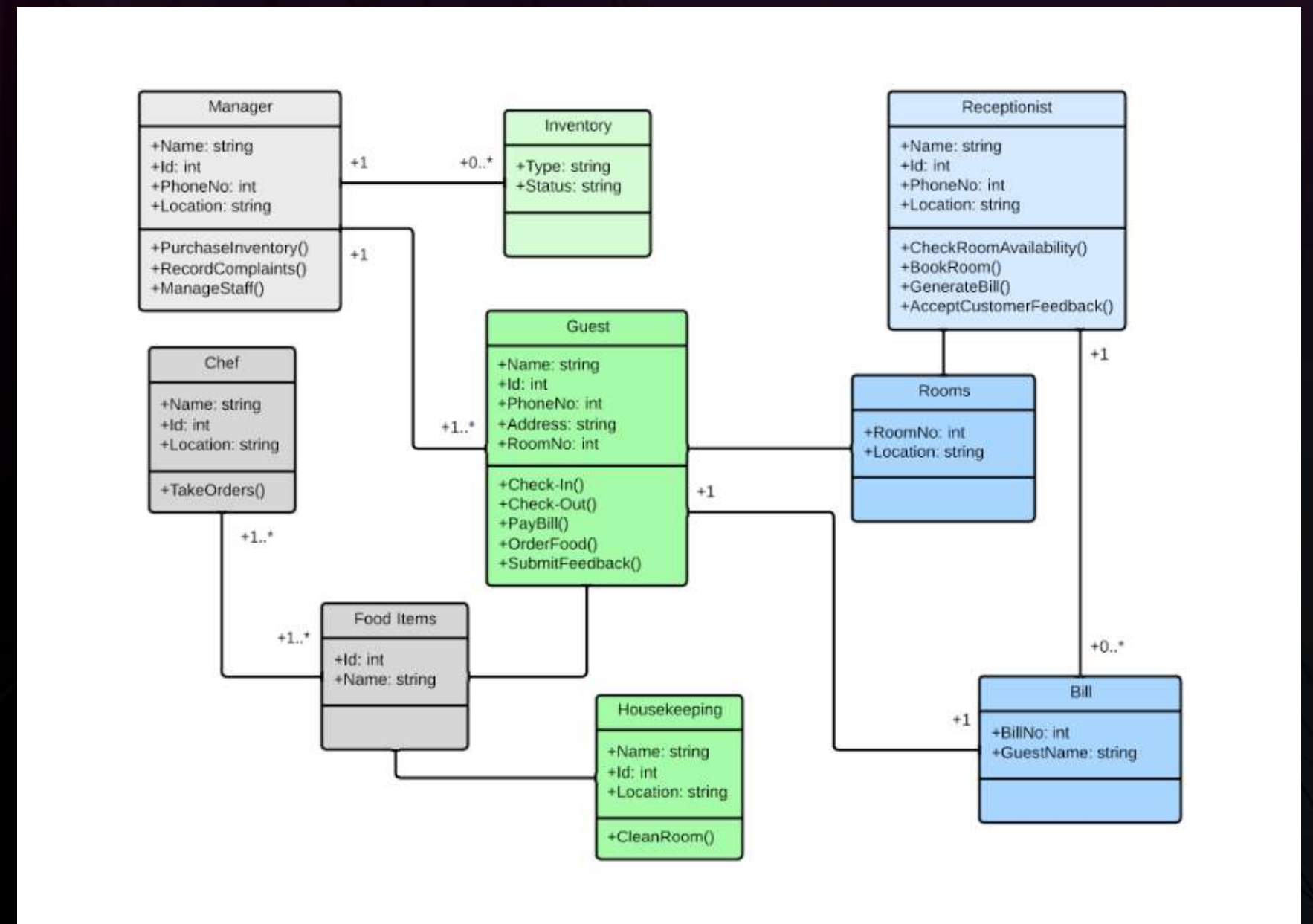
Una clase es un elemento de modelado que define las características del objeto que representa, incluidos sus atributos y comportamientos.

CLASES

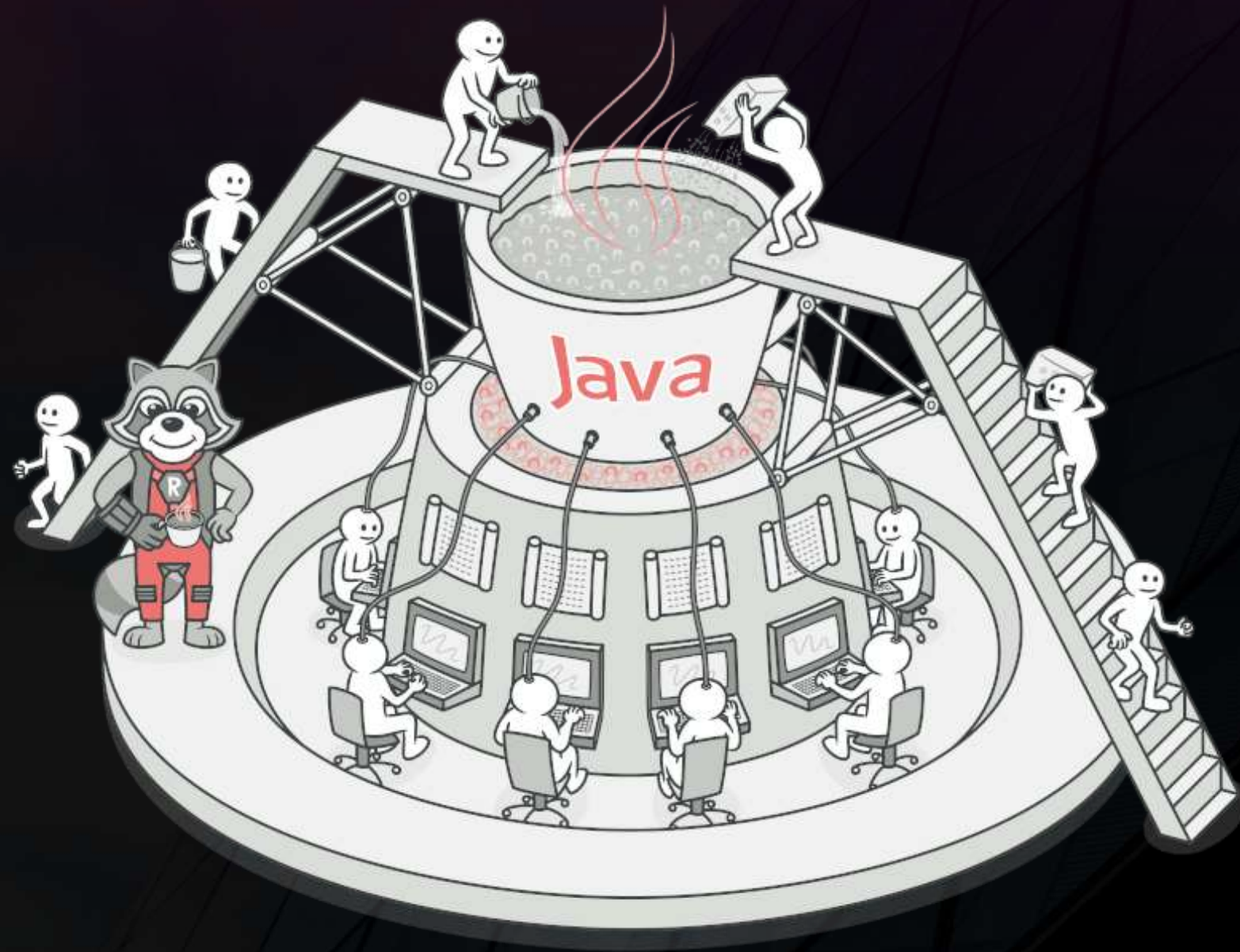
NOMBRE
ATRIBUTOS
METODOS
VISIBILIDAD

RELACIONES

ASOCIACION
AGREGACION Y COMPOSICION
HERENCIA
MULTIPLICIDAD



¿QUE SON LOS PATRONES DE DISEÑO?



Son soluciones habituales a problemas que ocurren con frecuencia en el diseño de software

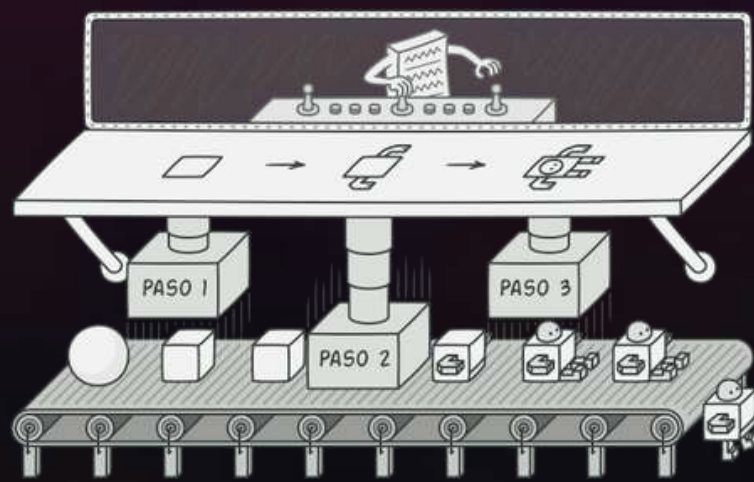
Son como planos que se pueden personalizar para resolver un problema de diseño

Son reutilizables y se pueden extraer, explicar en múltiples ámbitos



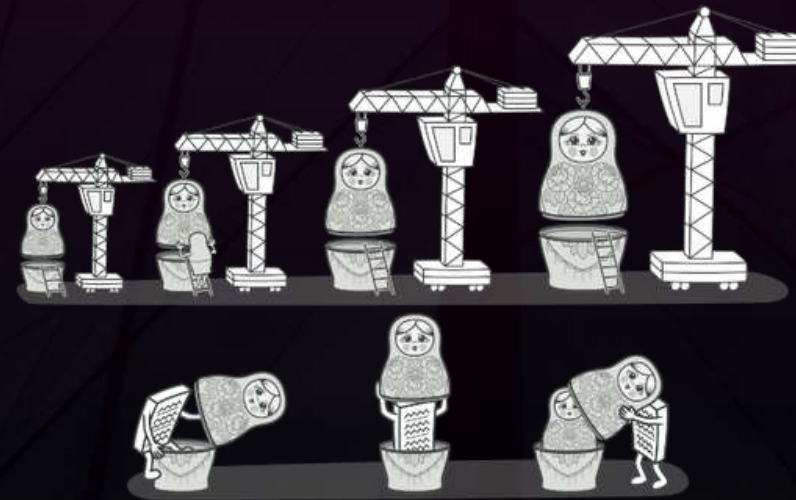
DOSW COMPANY

CATEGORIAS



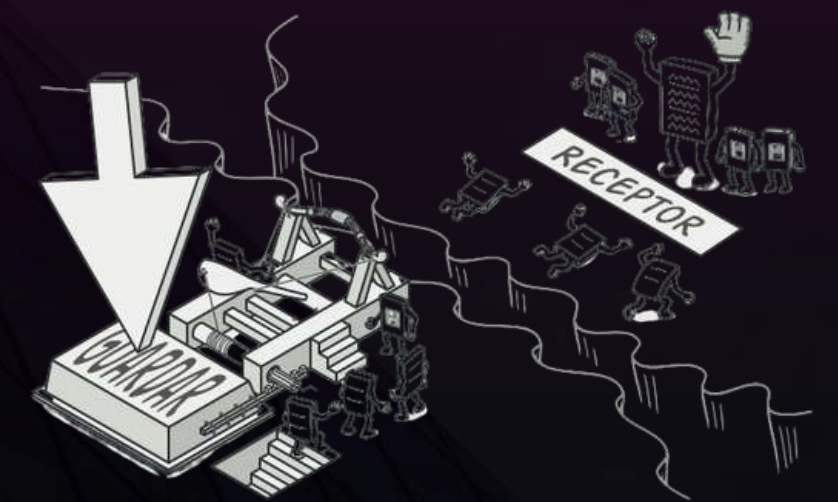
CREACIONALES

Proporcionan diversos mecanismos de creacion de objetos que incrementan la flexibilidad y reutilizacion del codigo existente



ESTRUCTURALES

Explican como ensamblar objetos y clases en estructuras mas grandes, mientras se mantiene la flexibilidad y eficiencia de la estructura

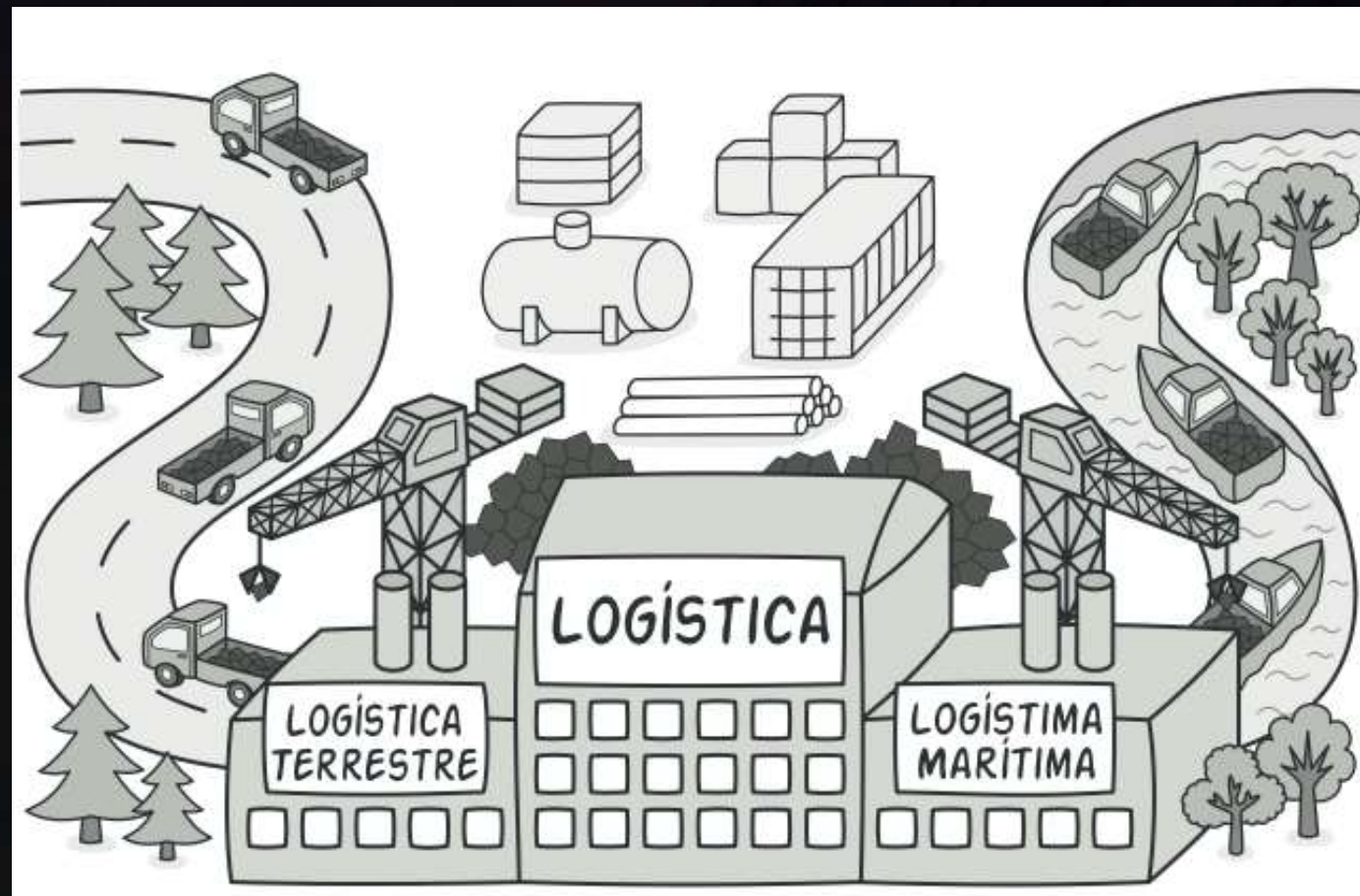


COMPORTAMIENTO

Estos patrones tratan con algoritmos y la asignacion de responsabilidades entre objetos, es decir como interactuan entre si

CREACIONAL - FACTORY METHOD

Es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.



Con el evitas un acoplamiento fuerte entre el creador y los productos concretos



Se presenta el principio de SINGLE RESPONSABILITY ya que puedes mover el código de la creación de un producto a un lugar del programa haciéndolo más fácil de mantener

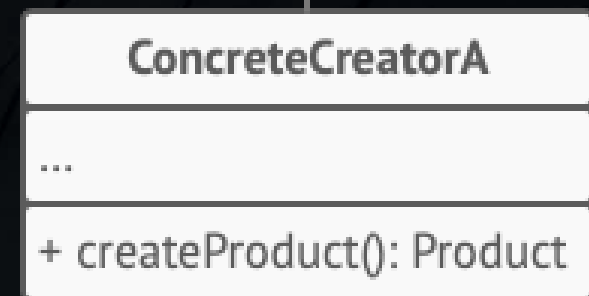
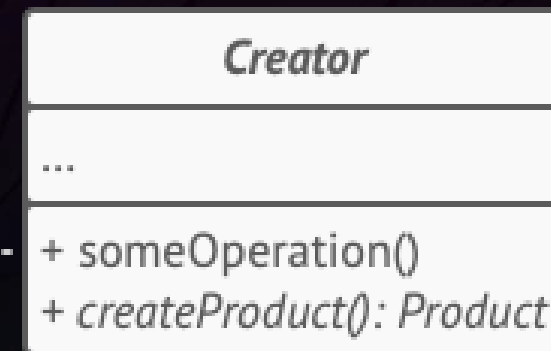


Se presenta el principio de OPEN/CLOSE al incorporar nuevos tipos de productos en el programa sin descomponer el código cliente existente

COMO IMPLEMENTARLO

1. La clase creadora declara el metodo fabrica que devuelve nuevos objetos de producto, el retorno debe coincidir con la interfaz de producto

```
Product p = createProduct()  
p.doStuff()
```



```
return new ConcreteProductA()
```

SE PUEDE USAR EL CREADOR COMO ABSTRACTO para forzar a las subclases a implementar sus propias versiones con overriding

1. Generen una interfaz que es comun para todos los objetos que puede producir la clase creadora y sus implementaciones concretas



1. Los creadores concretos sobrescriben el Factory Method base y devuelven un tipo diferente de producto

EJERCICIO #1



Una plataforma de comercio electrónico procesa pagos usando distintos métodos de pago:

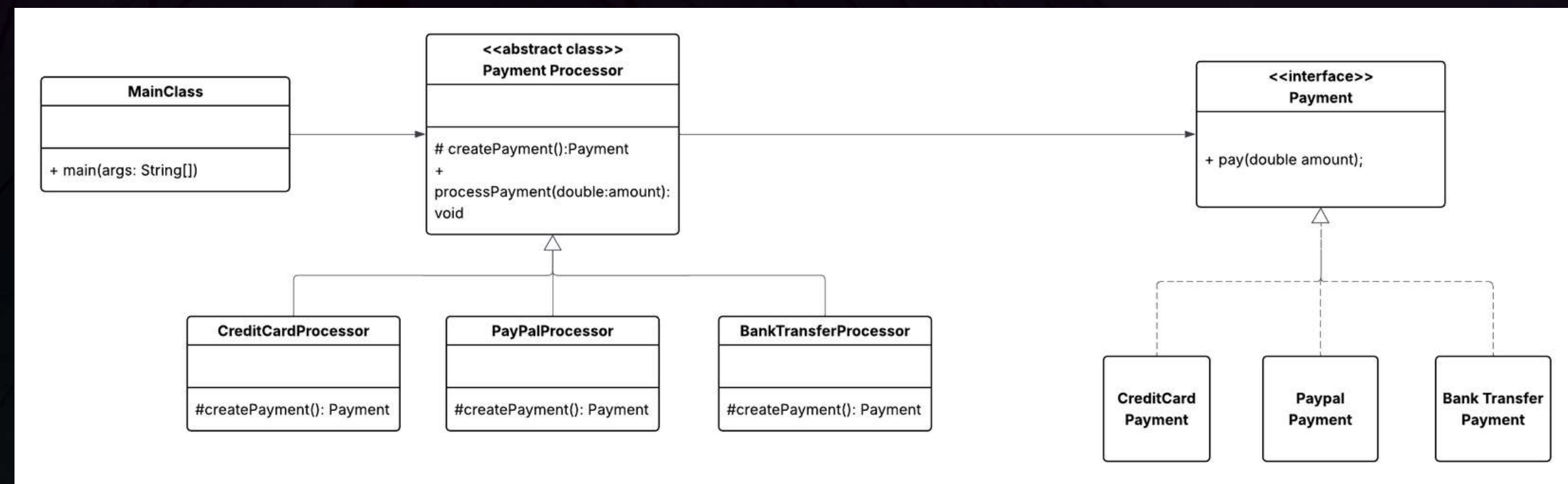
- Tarjeta de crédito
- PayPal
- Transferencia bancaria

El sistema no debe acoplarse a las clases concretas de cada tipo de pago.

Los pagos son con decimales

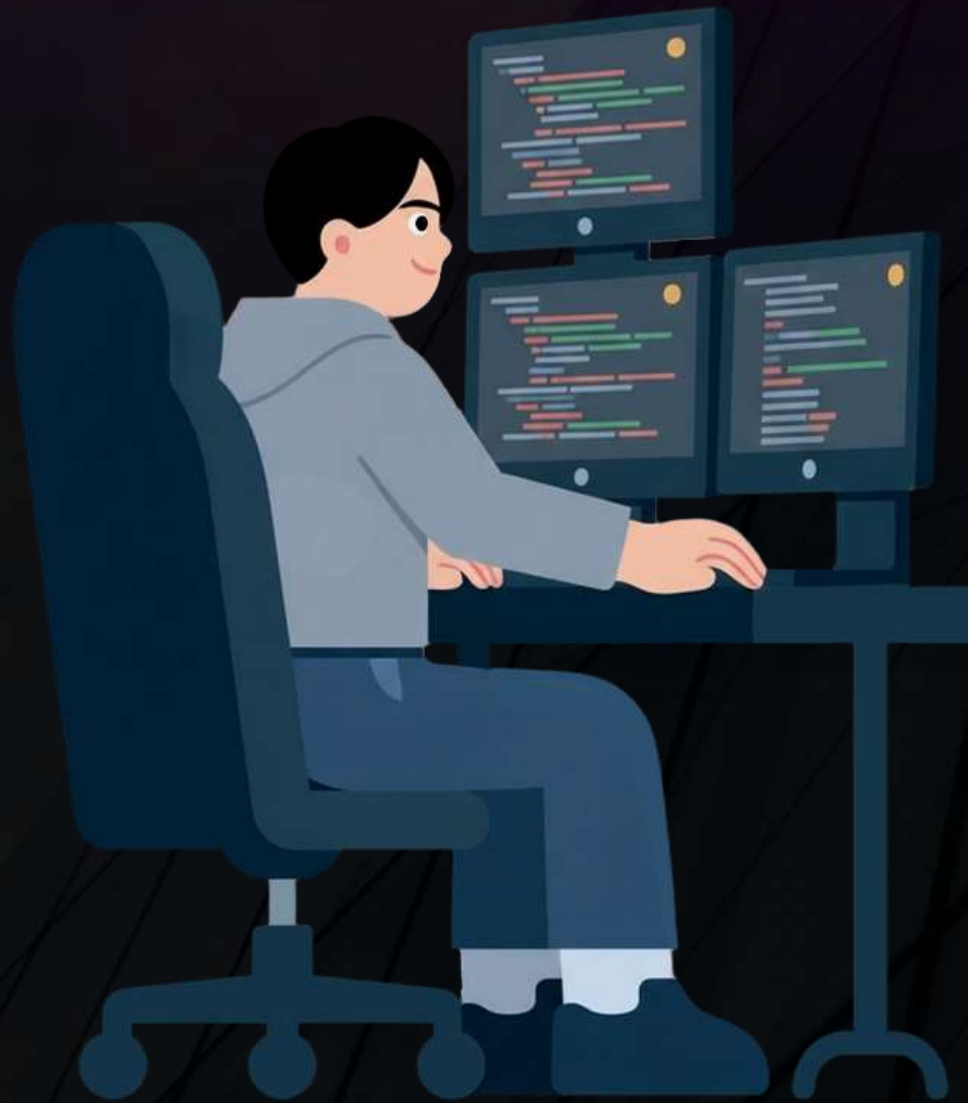
Cada pago debe decir:

Pago con METODO por \$ MONTO



EJERCICIO #1

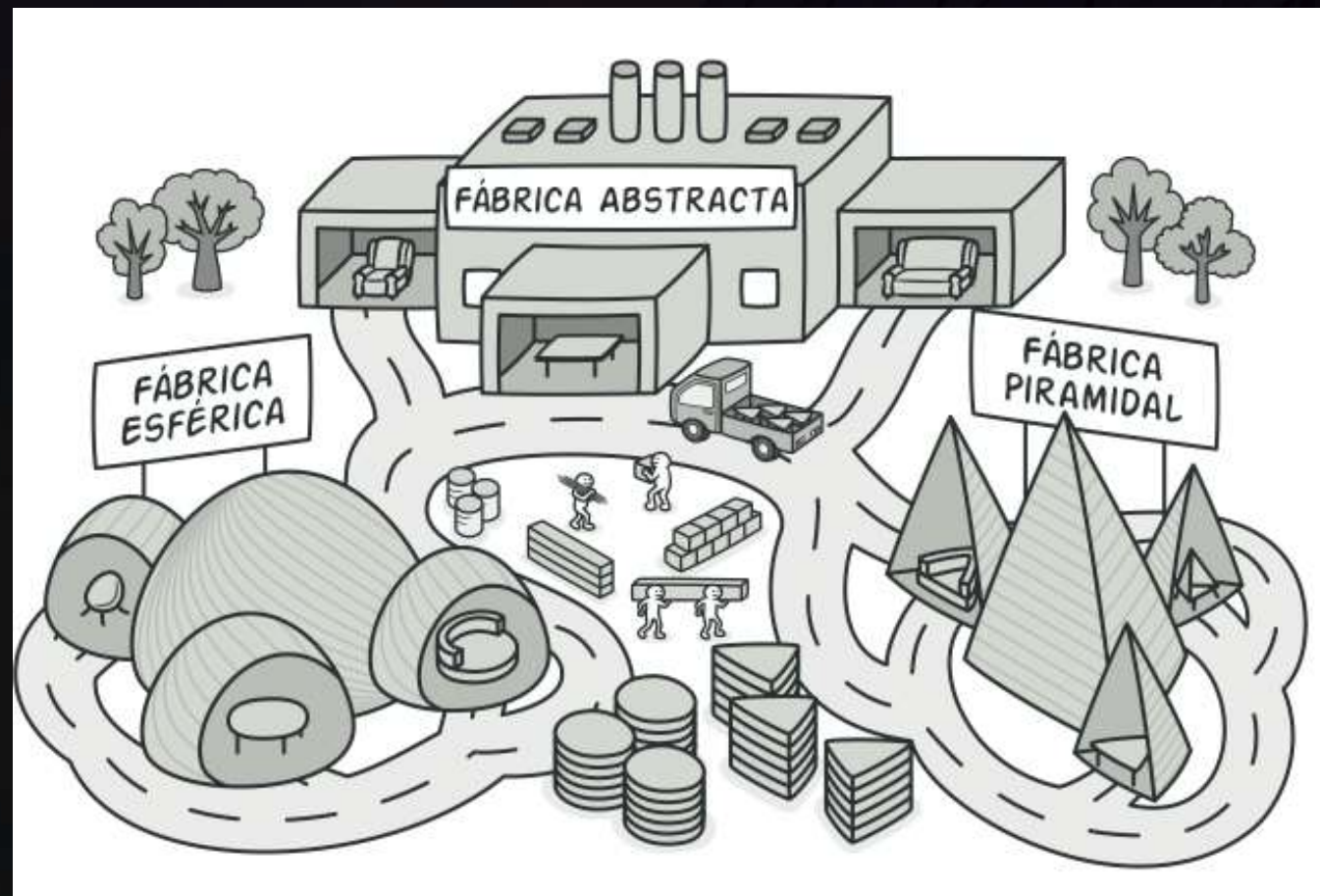
En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class Main {  
    public static void main(String[] args) {  
  
        PaymentProcessor processor;  
  
        processor = new CreditCardProcessor();  
        processor.processPayment(100);  
  
        processor = new PaypalProcessor();  
        processor.processPayment(250);  
  
        processor = new BankTransferProcessor();  
        processor.processPayment(500);  
    }  
}
```


CREACIONAL - ABSTRACT FACTORY

Es un patrón de diseño creacional que nos permite producir familias de objetos relacionados sin especificar sus clases concretas.



✓ Puedes tener la certeza de que los productos que obtienes en una fábrica son compatibles entre si

✓ Evitas un acoplamiento fuerte entre productos y código cliente

✓ Se presenta el principio de SINGLE RESPONSABILITY ya que puedes mover el código de la creación de un producto a un lugar del programa haciéndolo más fácil de mantener

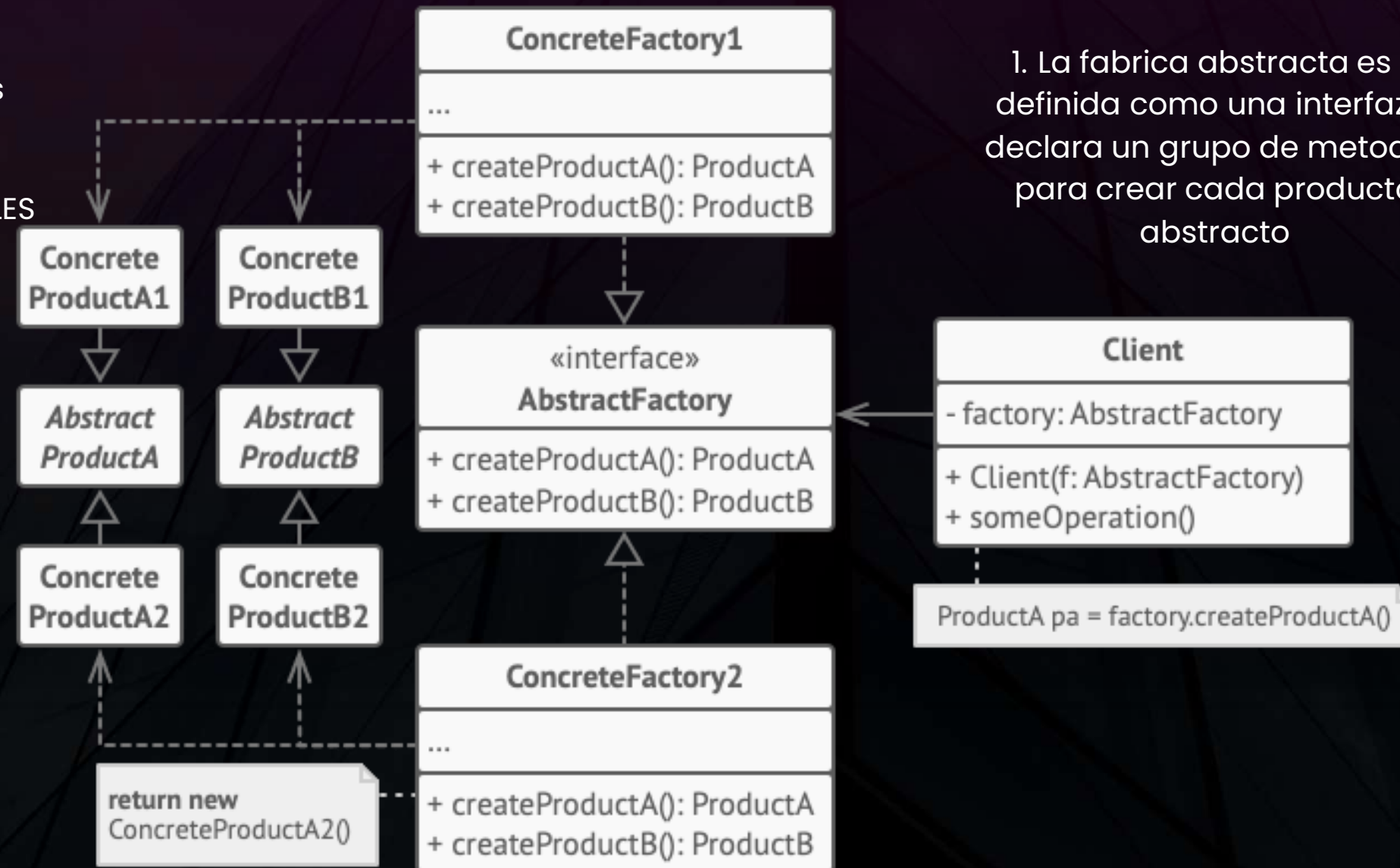
✓ Se presenta el principio de OPEN/CLOSE al incorporar nuevos tipos de productos en el programa sin descomponer el código cliente existente

COMO IMPLEMENTARLO

1. Los productos concretos son implementaciones distintas de los abstractos agrupados por variantes

CADA PRODUCTO ABSTRACTO DEBE IMPLEMENTARSE EN TODAS LAS VARIABLES DADAS

1. Los productos abstractos declaran interfaces para un grupo de productos diferentes pero relacionados en familia



1. La fabrica abstracta es definida como una interfaz y declara un grupo de metodos para crear cada producto abstracto

1. El codigo cliente no se acopla a la variante del producto y puede funcionar como cualquier fabrica

1. Las fabricas concretas implementan metodos de creacion de la fabrica abstracta, cada fabrica es una variante de los productos

LAS FIRMAS DE LOS METODOS DE CREACION DEBEN DEVOLVER LOS PRODUCTOS ABSTRACTOS CORRESPONDIENTES



EJERCICIO #2

Una empresa desarrolla videojuegos que deben ejecutarse en distintas consolas. Cada consola ofrece una familia de componentes compatibles entre sí:

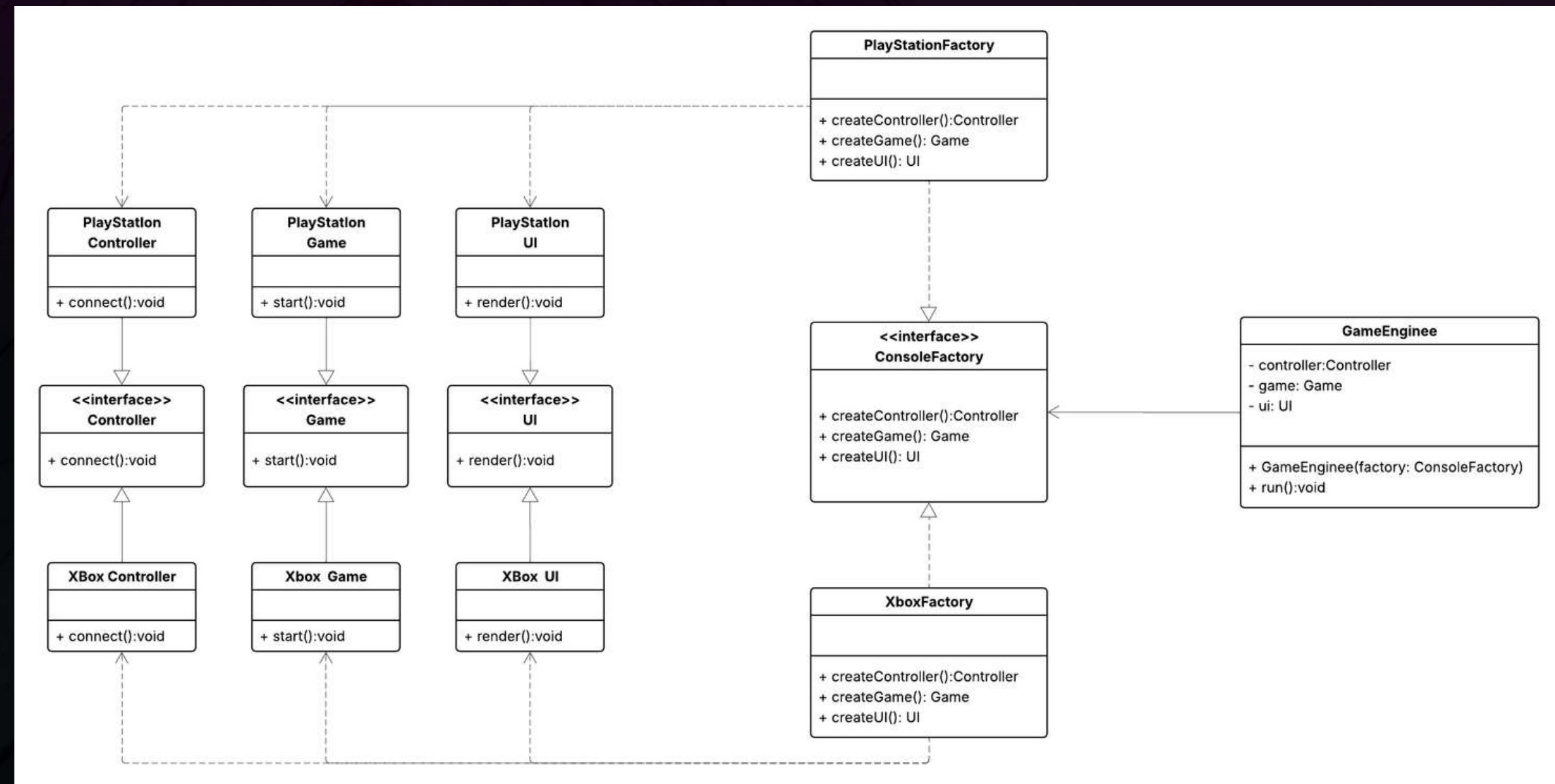
- Control
- Juego
- Interfaz gráfica

Las consolas soportadas inicialmente son:

- PlayStation
- Xbox

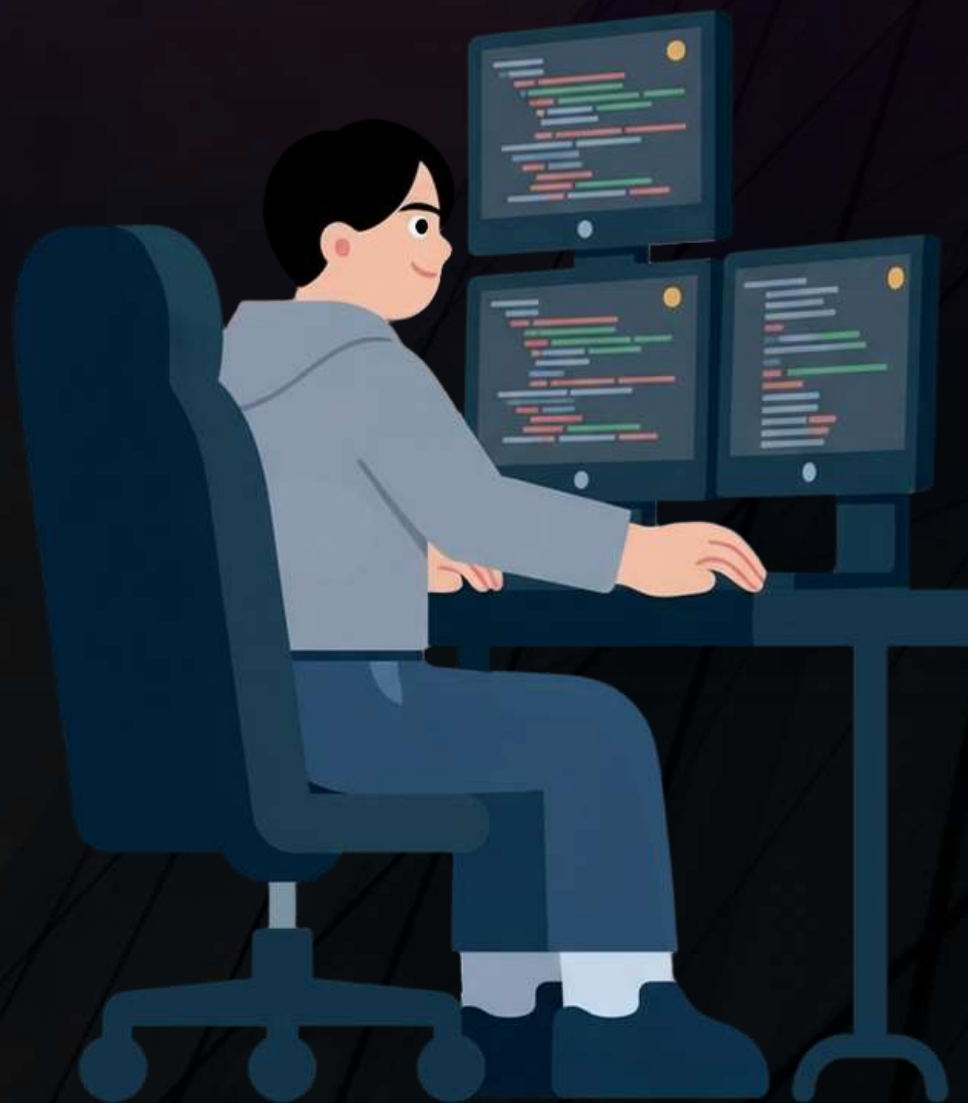
El motor del juego no debe conocer las implementaciones concretas de estos componentes.

- El control puede conectarse
- El juego iniciarse
- Y la interfaz grafica renderizar



EJERCICIO #2

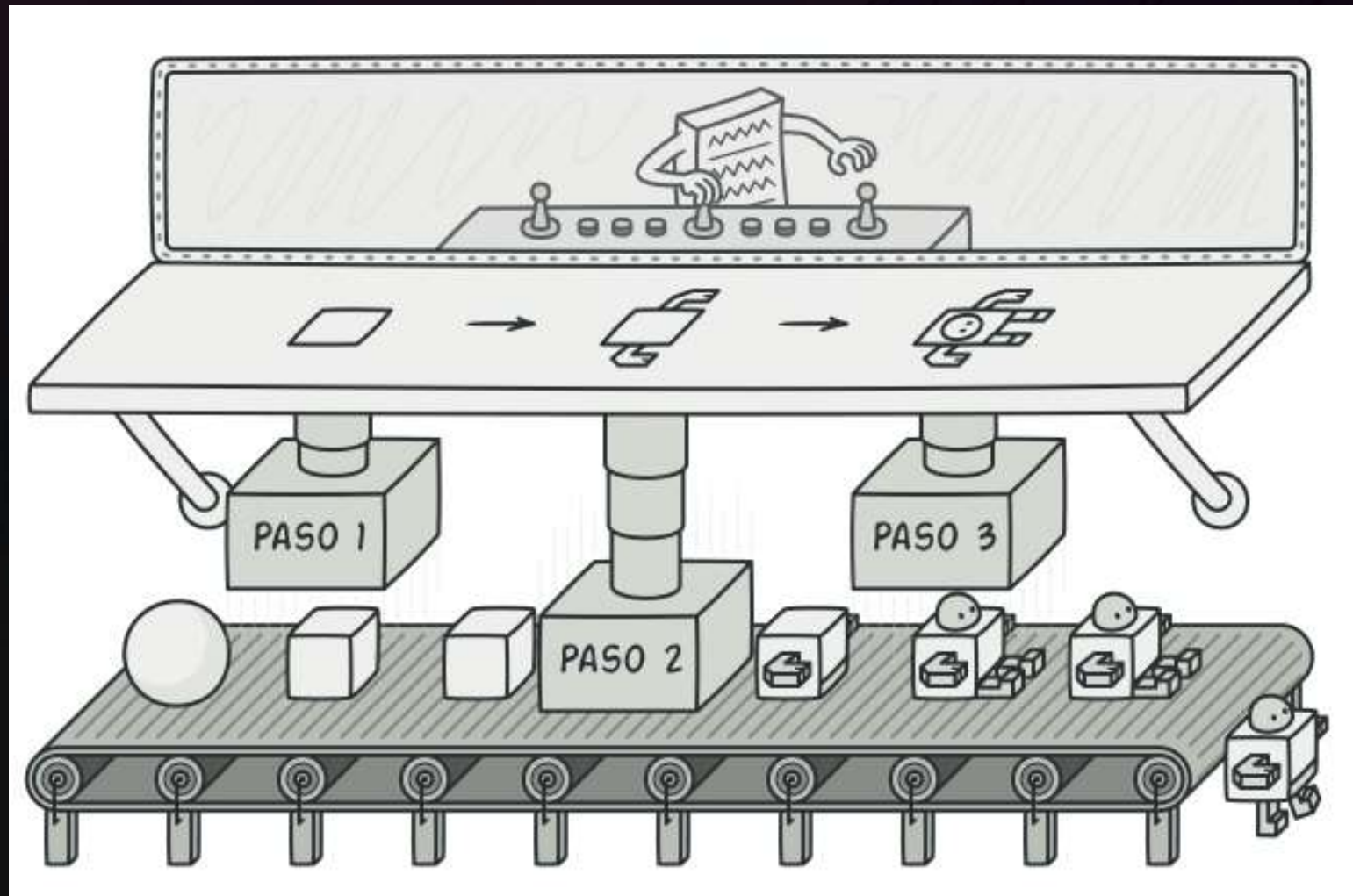
En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class Main {  
    public static void main(String[] args) {  
  
        ConsoleFactory factory;  
  
        factory = new PlayStationFactory();  
        GameEngine psEngine = new GameEngine(factory);  
        psEngine.run();  
  
        System.out.println("-----");  
  
        factory = new XboxFactory();  
        GameEngine xboxEngine = new GameEngine(factory);  
        xboxEngine.run();  
    }  
}
```

CREACIONAL - BUILDER

Es un patrón de diseño creacional que nos permite construir objetos complejos paso a paso. El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.



Puedes construir objetos paso a paso, aplazar pasos de la construcción o ejecutar pasos de forma recursiva



Puedes reutilizar el mismo código de construcción al construir varias representaciones de productos



Utiliza SINGLE RESPONSIBILITY al aislar el código de construcción complejo de la lógica de negocio del producto



No es necesario crear constructores tan grandes o especializados dependiendo la combinación de los datos de entrada

COMO IMPLEMENTARLO

1. La interfaz constructora declara pasos de construccion de producto que todos los objetos constructores tienen en comun

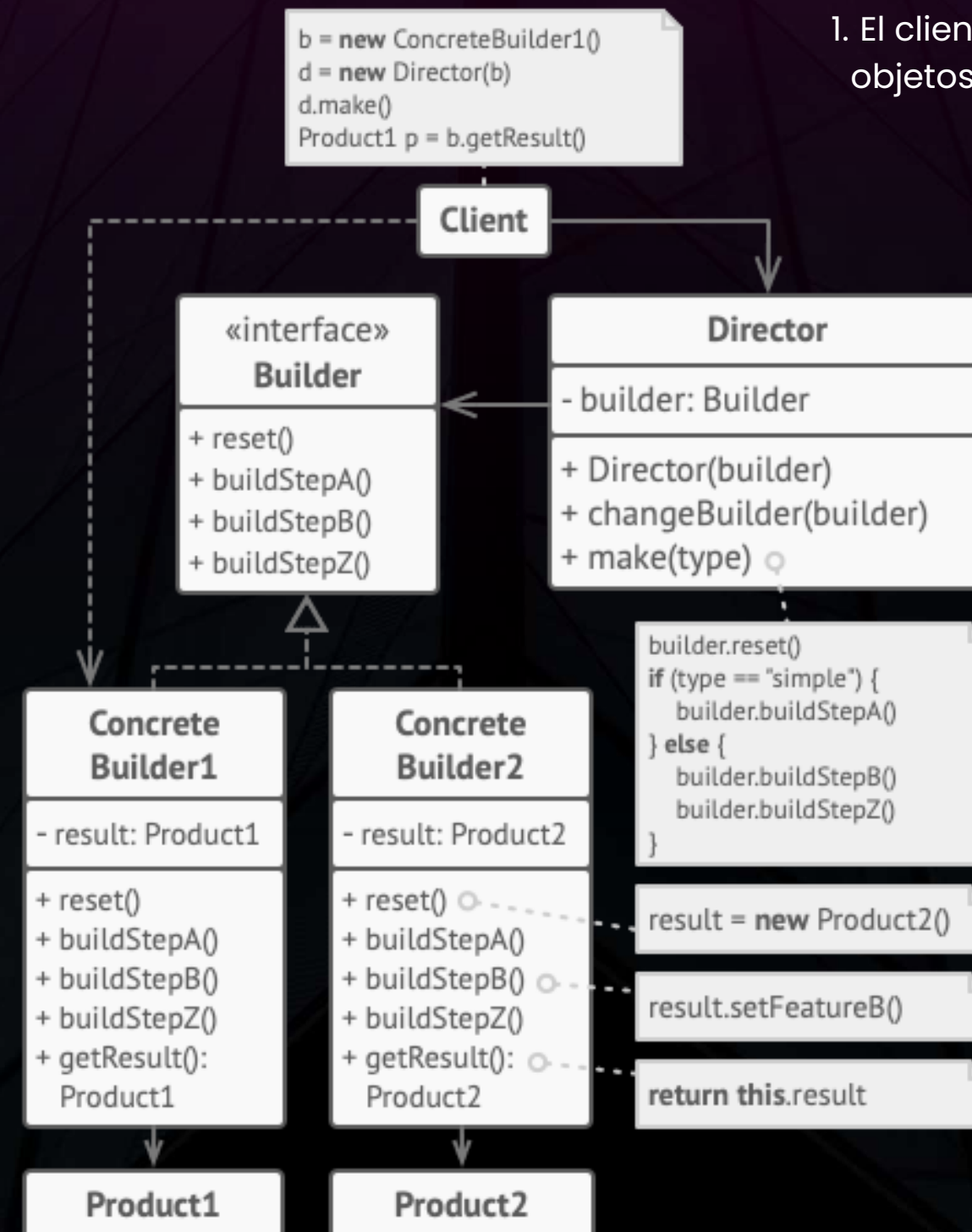
1. Los constructores concretos ofrecen distintas implementaciones de los pasos de la construccion y crear productos que no siguen la interfaz comun

1. Los productos son los objetos resultantes, no tienen que perteneces a la misma jerarquia de clases o interfaz

1. El cliente debe asociar uno de los objetos constructores con la clase directora

1. El director define el orden en el que se invocan los pasos de construccion por lo que puedes crear y reutilizar configuraciones especificas de los productos

EL BUILDER NO ELIMINA LOS PARAMETROS
LOS CONVIERTE EN PASOS EXPLICITOS Y
CON NOMBRE



EJERCICIO #3

Una fábrica de juguetes produce muñecos con distintas configuraciones.

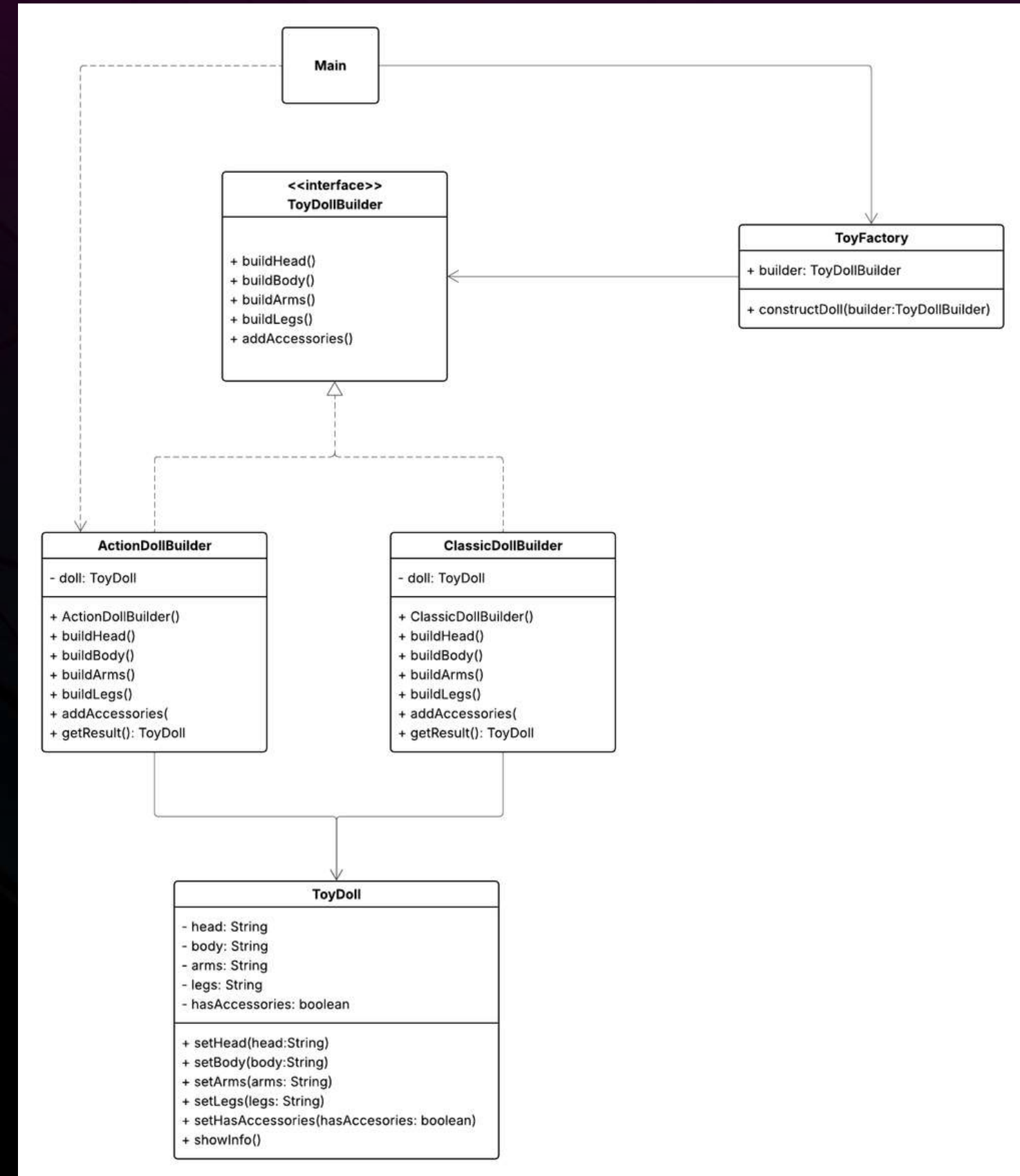
El proceso de ensamblaje es el mismo, pero el resultado final varía según el tipo de muñeco solicitado.

Ejemplos de muñecos:

- Muñeco de acción
- Muñeca clásica

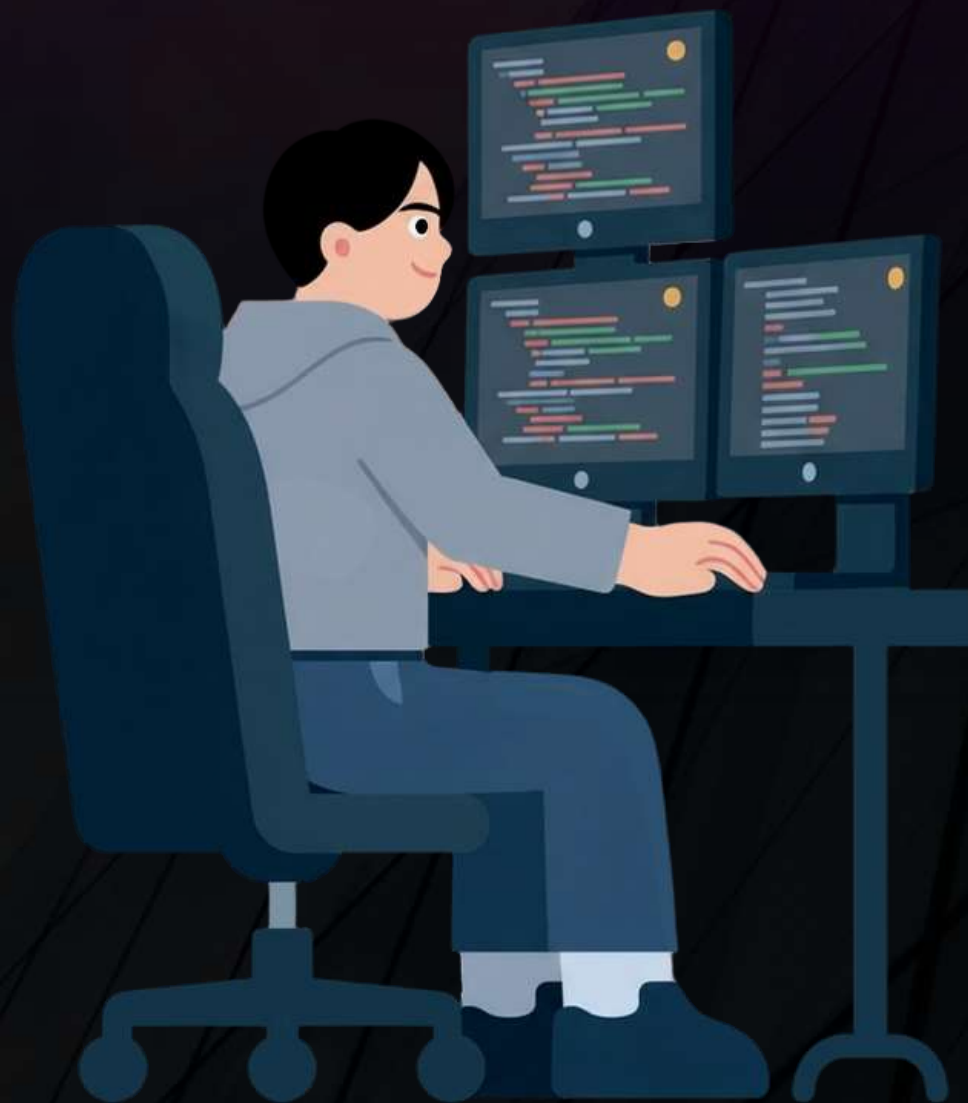
La fábrica quiere separar el proceso de construcción del objeto final.

Cada muñeco esta conformado de cabeza, cuerpo, brazos, piernas y puede o no tener accesorios



EJERCICIO #3

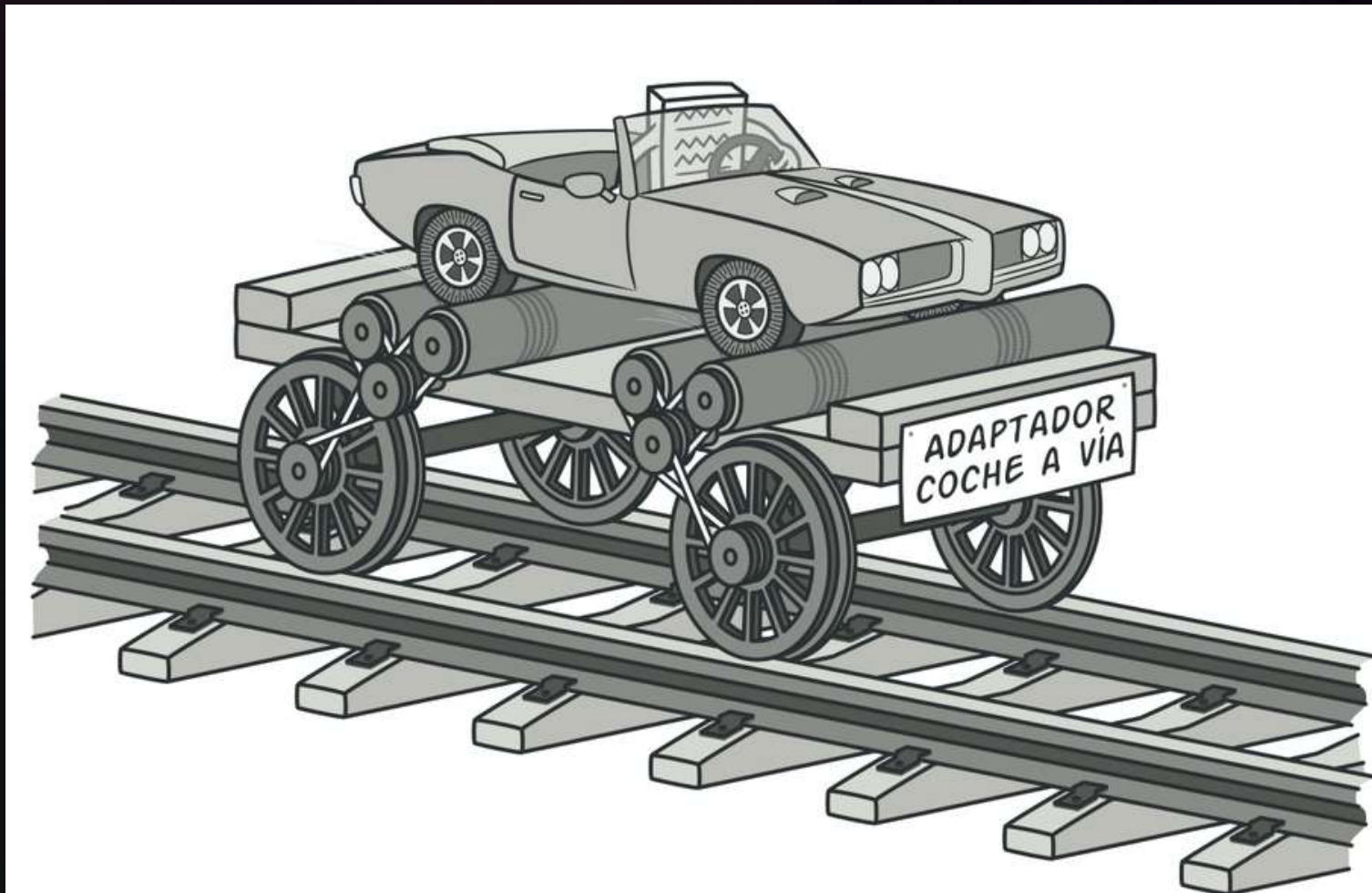
En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class Main {  
    public static void main(String[] args) {  
  
        ToyFactory factory = new ToyFactory();  
  
        ActionDollBuilder actionBuilder = new ActionDollBuilder();  
        factory.constructDoll(actionBuilder);  
        ToyDoll actionDoll = actionBuilder.getResult();  
  
        ClassicDollBuilder classicBuilder = new ClassicDollBuilder();  
        factory.constructDoll(classicBuilder);  
        ToyDoll classicDoll = classicBuilder.getResult();  
  
        actionDoll.showInfo();  
        classicDoll.showInfo();  
    }  
}
```


ESTRUCTURAL - ADAPTER

Es un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles.



El patrón Adapter te permite crear una clase intermedia que sirva como traductora entre tu código y una clase heredada, una clase de un tercero o cualquier otra clase con una interfaz extraña.

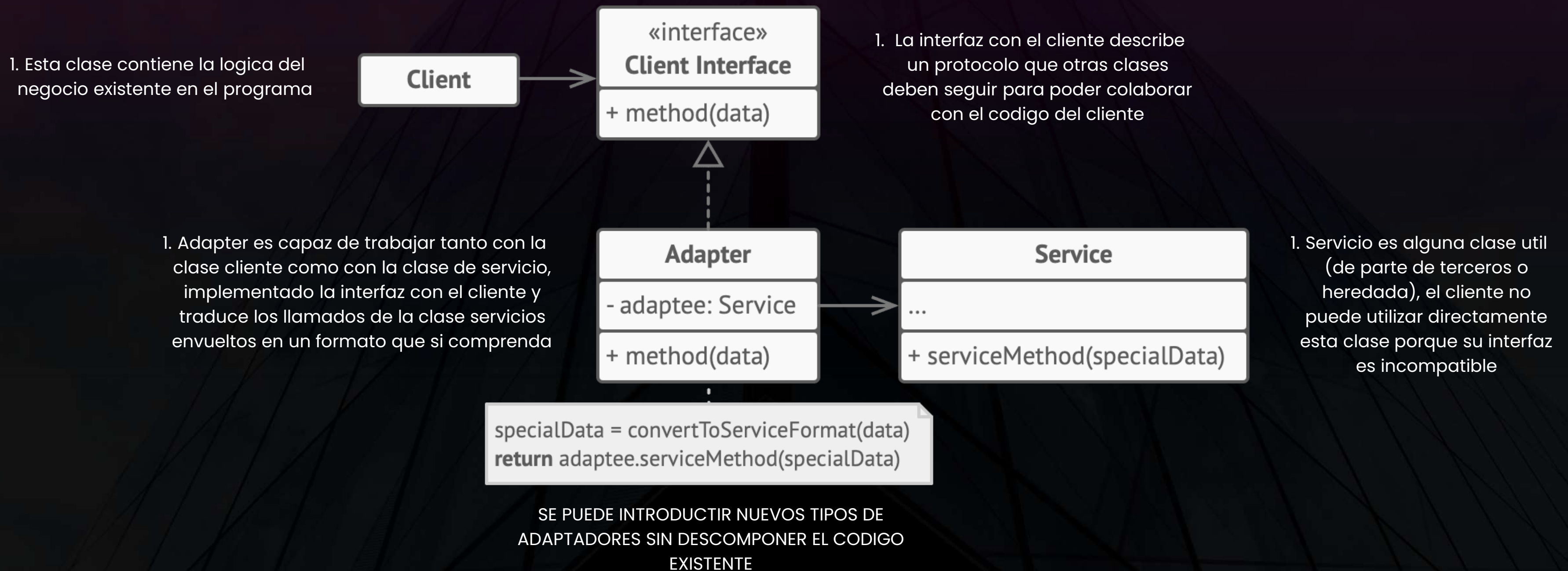


Se utiliza el SINGLE RESPONSABILITY ya que puedes separar la interfaz o el código de conversión de datos de la lógica de negocio primaria del programa.



Se aplica principio de OPEN/CLOSE, ya que puedes introducir nuevos tipos de adaptadores al programa sin descomponer el código cliente existente, siempre y cuando trabajen con los adaptadores a través de la interfaz con el cliente.

COMO IMPLEMENTARLO

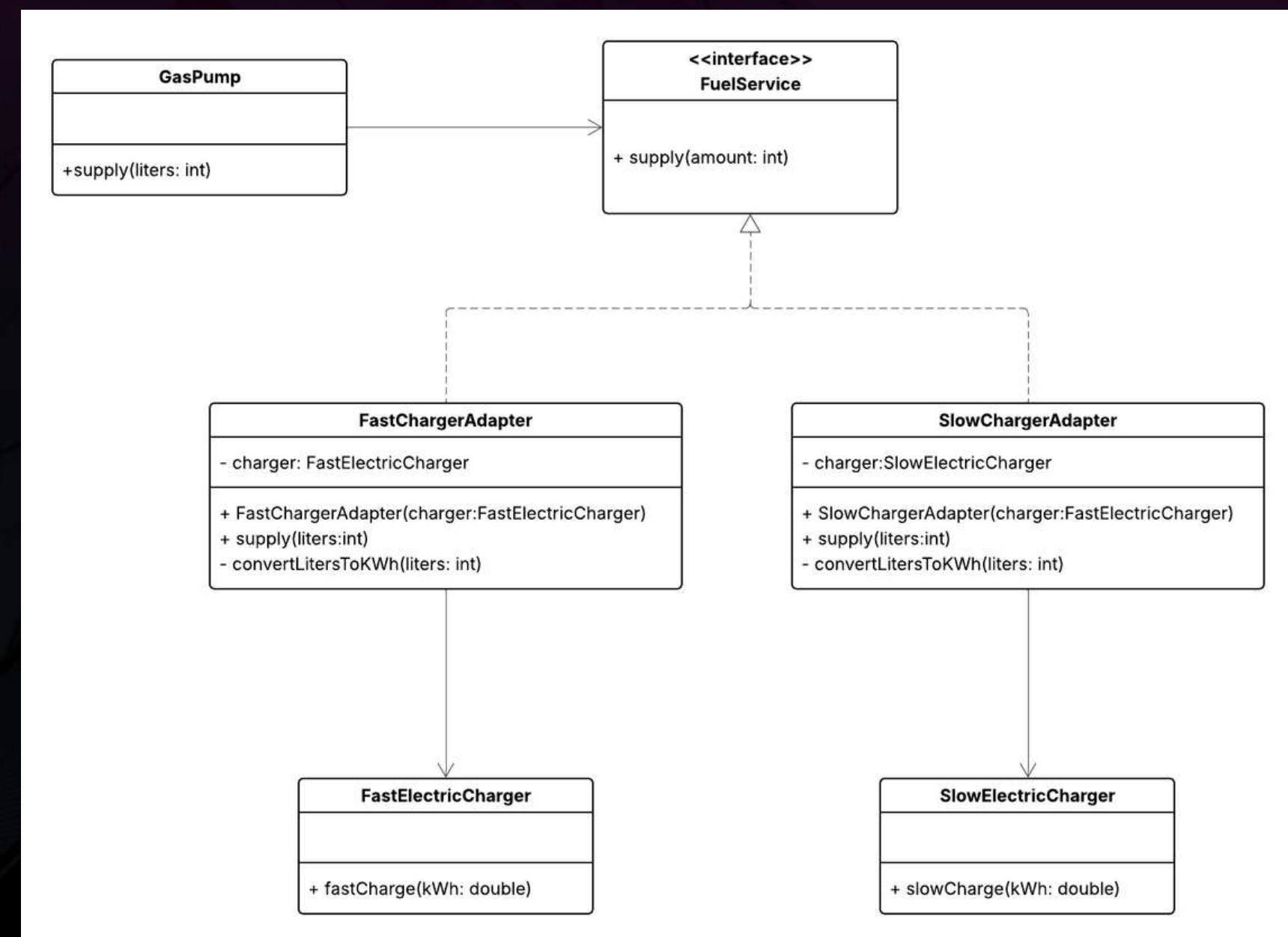




EJERCICIO #4

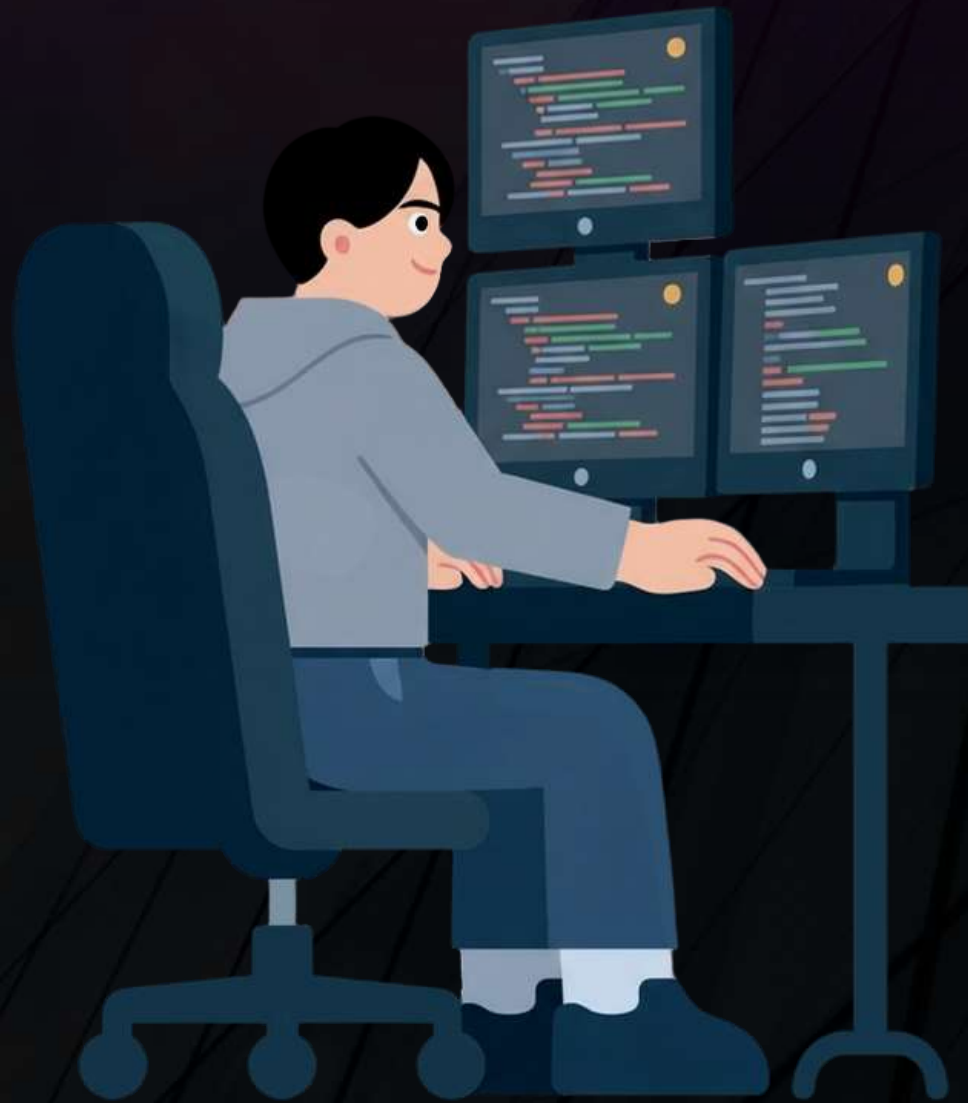
- Una gasolinera inteligente fue diseñada para atender vehículos a combustión usando un sistema estándar interno.
- Con la llegada de vehículos eléctricos e híbridos, la estación incorpora cargadores eléctricos de distintos proveedores, cada uno con interfaces incompatibles que no pueden modificarse.
- El sistema central de la gasolinera no debe cambiar.
- Unificar el proceso de abastecimiento para todos los vehículos, permitiendo que el sistema use una sola interfaz, aplicando el patrón Adapter.

Para convertir litros en KWh en el modelo Rapido se multiplica los litros * 8.0, en el modelo lento se multiplica los litros por * 7,0



EJERCICIO #4

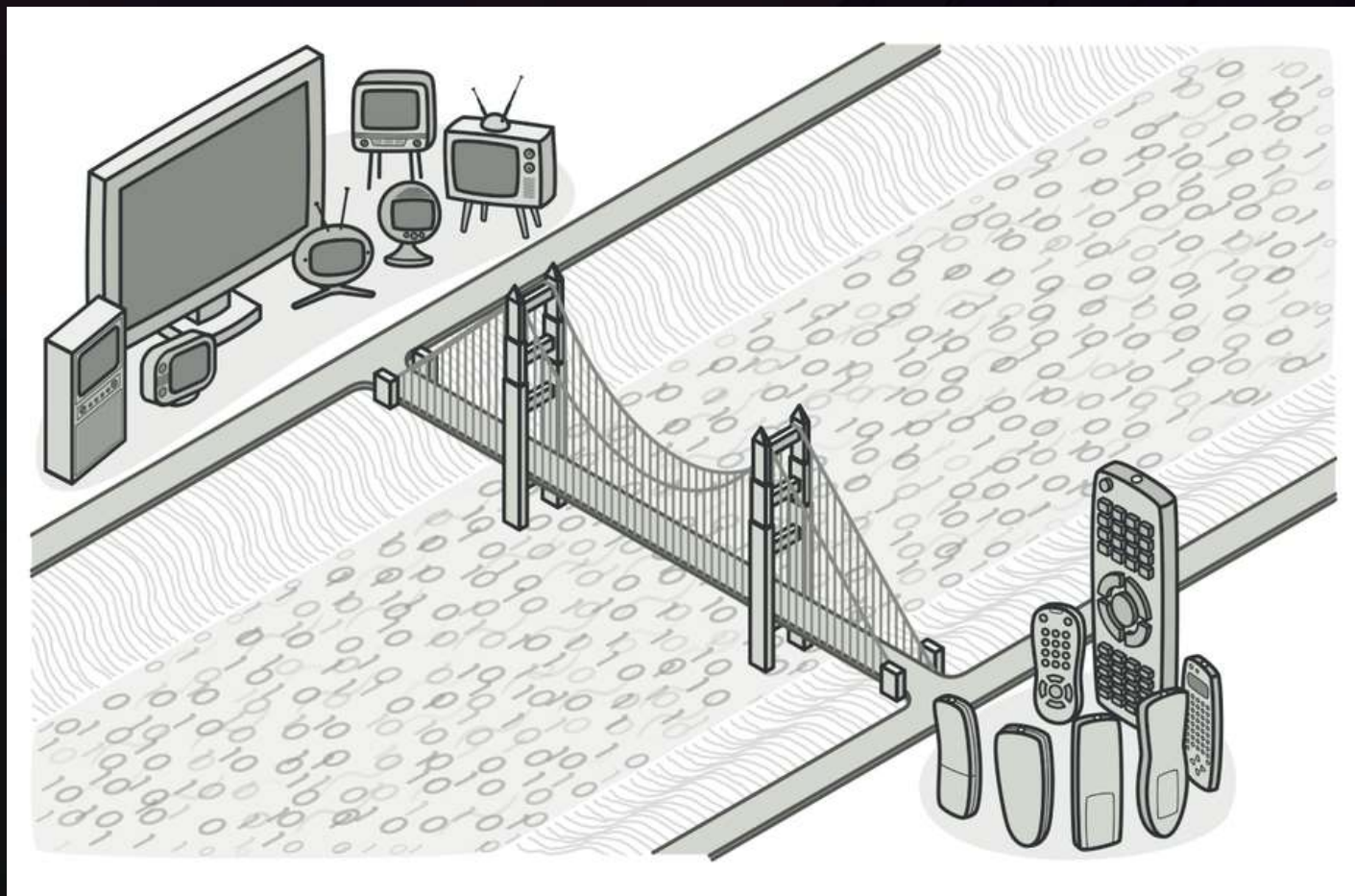
En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class SmartGasStation {  
  
    public static void main(String[] args) {  
  
        FuelService gasolinePump = new GasPump();  
  
        FuelService fastElectricPump =  
            new FastChargerAdapter(new FastElectricCharger());  
  
        FuelService slowElectricPump =  
            new SlowChargerAdapter(new SlowElectricCharger());  
  
        gasolinePump.supply(30);  
        fastElectricPump.supply(30);  
        slowElectricPump.supply(30);  
  
    }  
}
```

ESTRUCTURAL - BRIDGE

Es un patrón de diseño estructural que te permite dividir una clase grande, o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas (abstracción e implementación) que pueden desarrollarse independientemente la una de la otra.

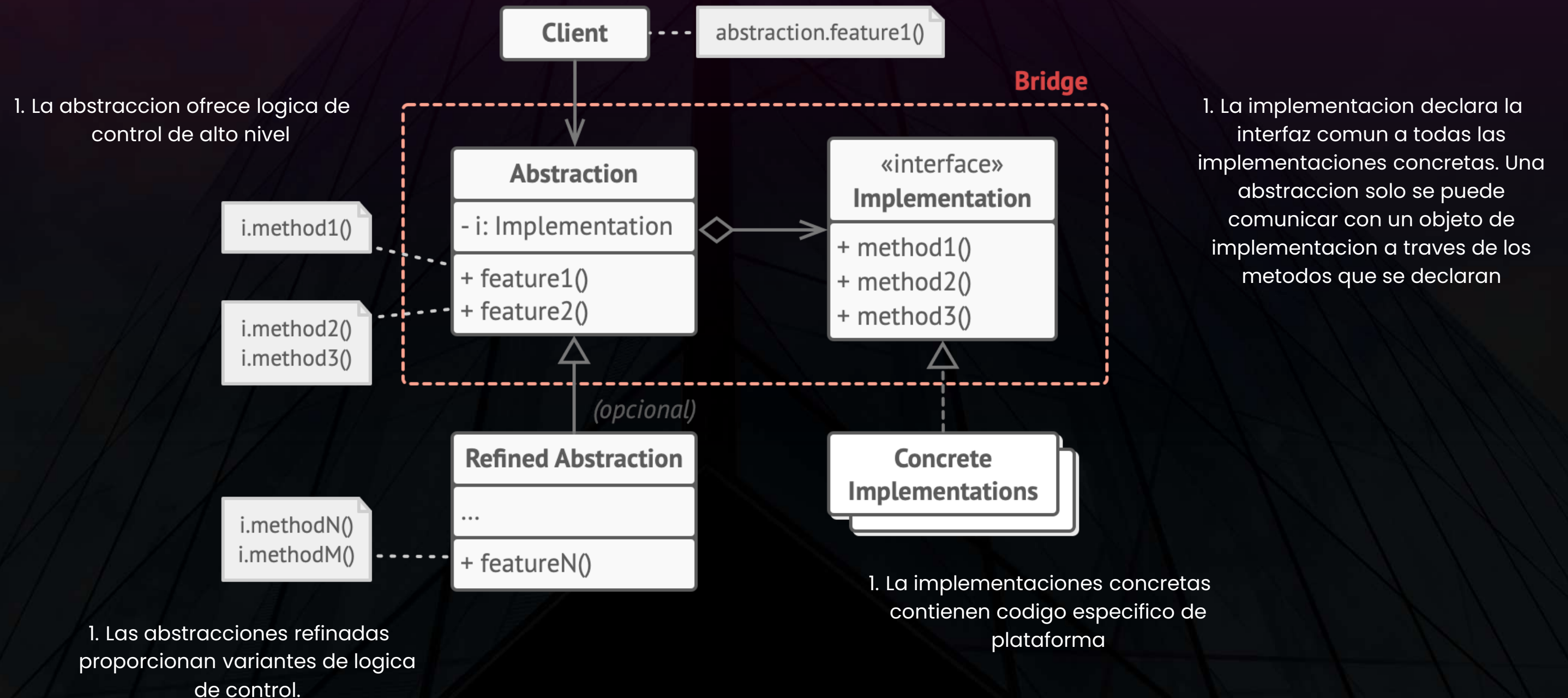


La Abstracción (también llamada interfaz) es una capa de control de alto nivel para una entidad. Esta capa no tiene que hacer ningún trabajo real por su cuenta, sino que debe delegar el trabajo a la capa de implementación (también llamada plataforma).

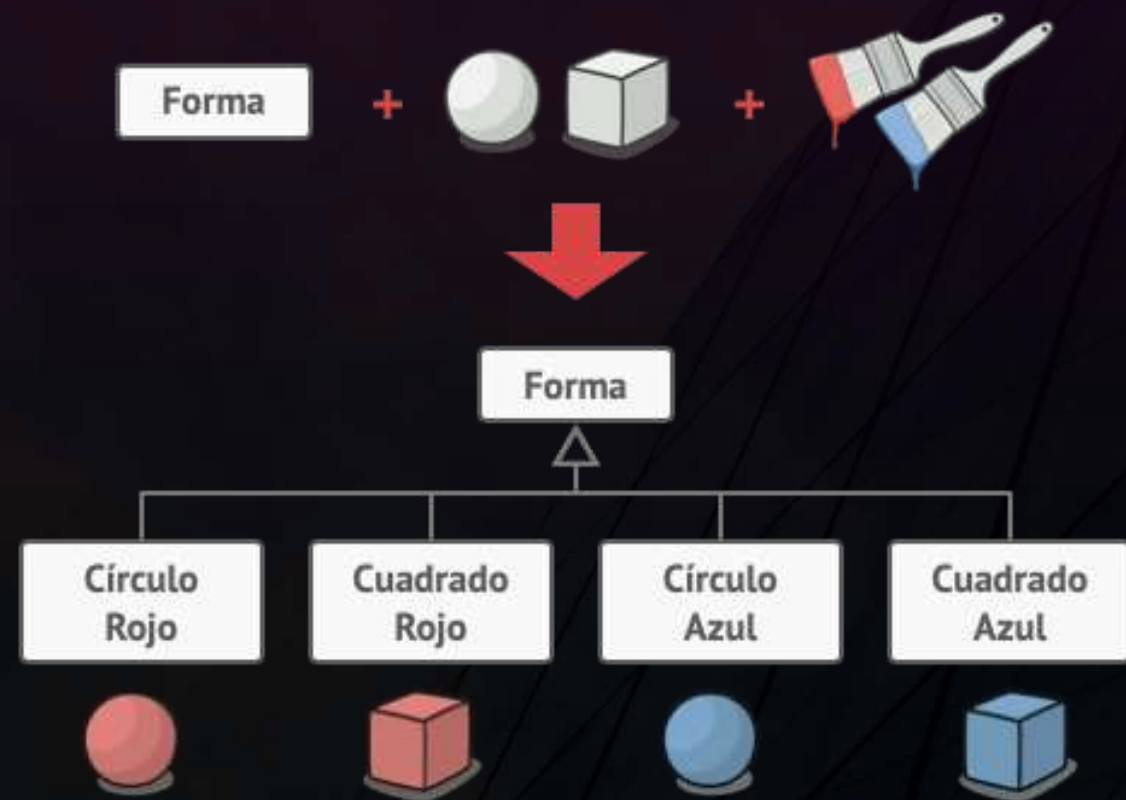


El código cliente funciona con abstracciones de alto nivel. No está expuesto a los detalles de la plataforma.

COMO IMPLEMENTARLO



EJERCICIO #5



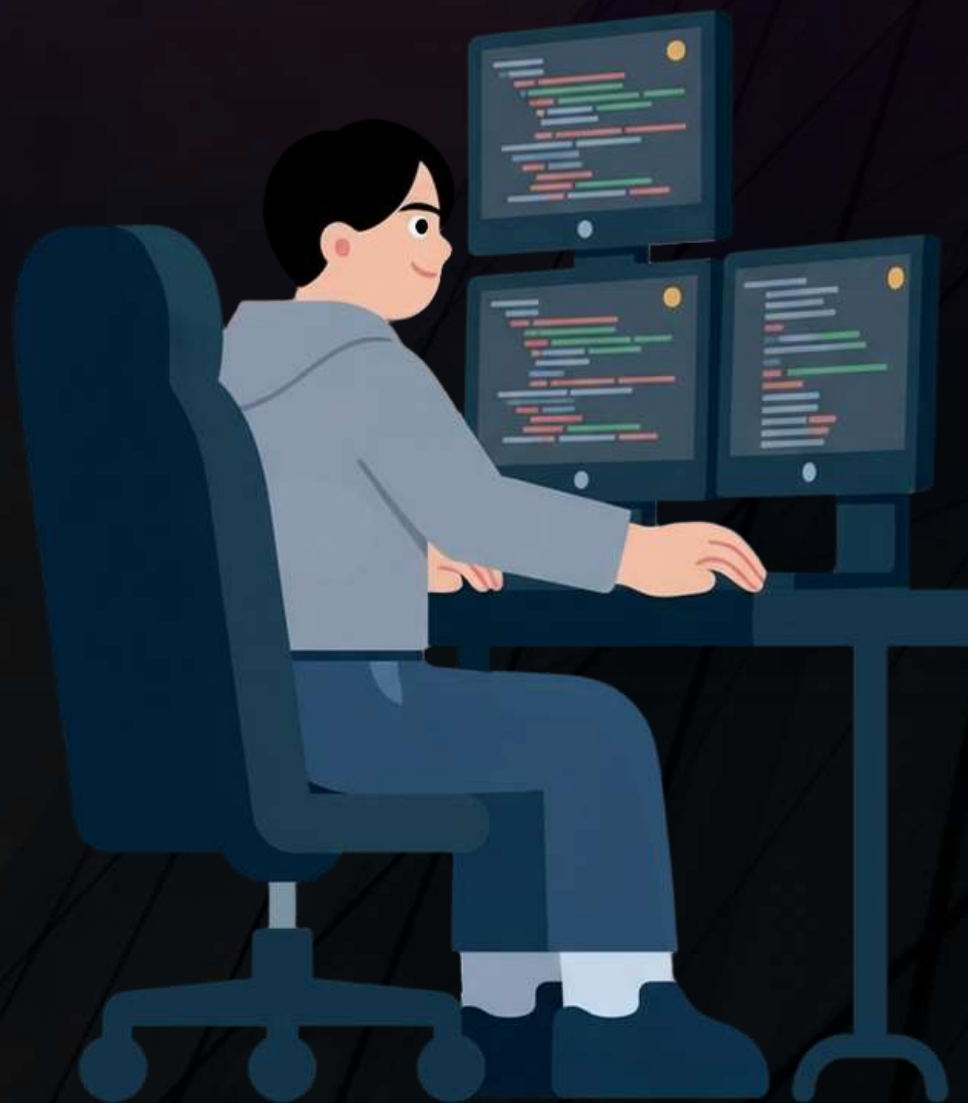
Se tiene una clase geometrica Forma con un par de subclases Círculo y Cuadrado, al extender esa jerarquía para que incorpore colores, toca crear las combinaciones de clases, lo que hace que aumente exponencialmente



El patrón Bridge intenta resolver este problema pasando de la herencia a la composición del objeto. Esto quiere decir que se extrae una de las dimensiones a una jerarquía de clases separada, de modo que las clases originales referencian un objeto de la nueva jerarquía, en lugar de tener todo su estado y sus funcionalidades dentro de una clase.

EJERCICIO #5

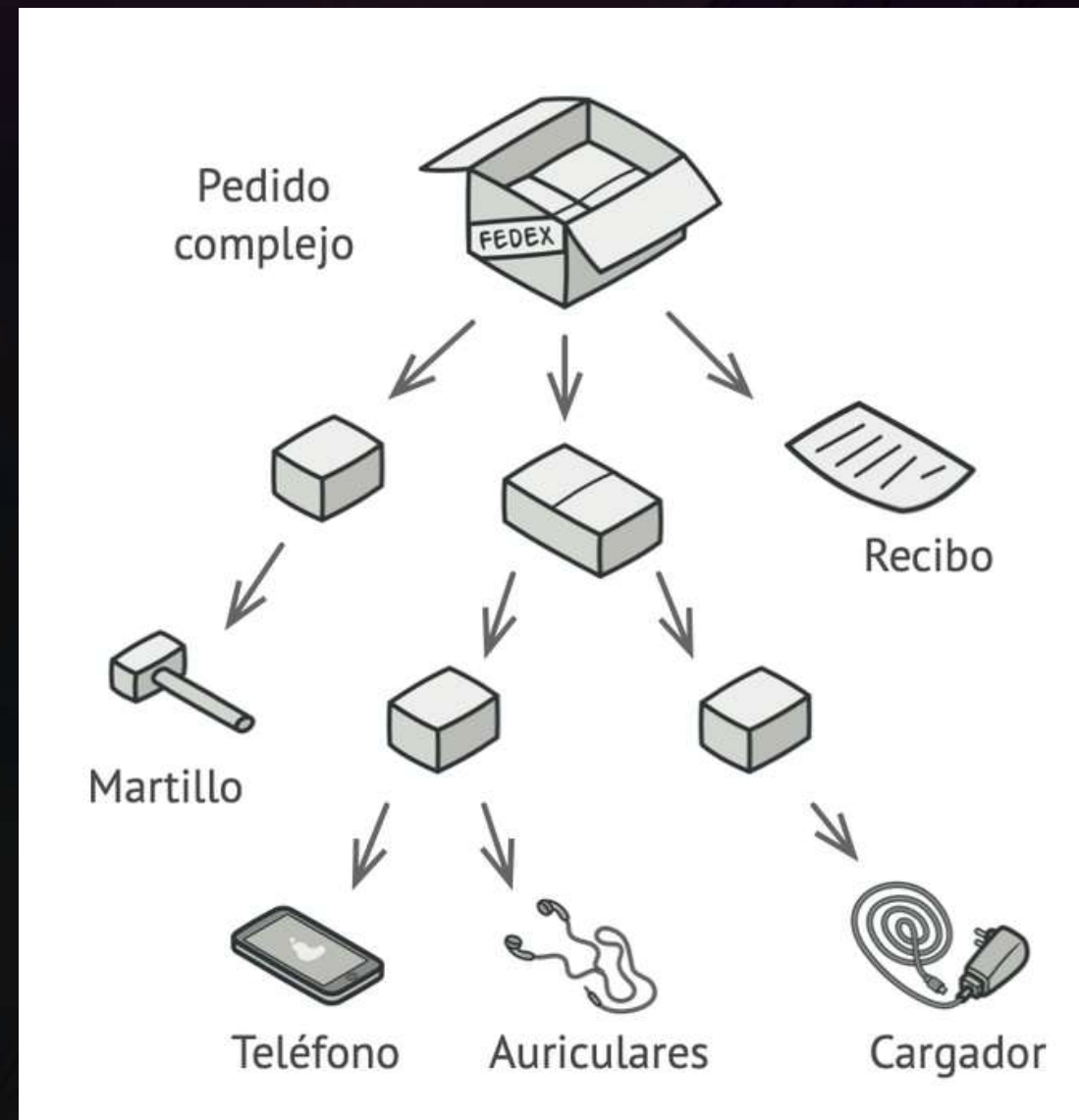
En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class Main {  
  
    public static void main(String[] args) {  
  
        Forma circuloRojo = new Circulo(new Rojo());  
        Forma cuadradoRojo = new Cuadrado(new Rojo());  
  
        Forma circuloAzul = new Circulo(new Azul());  
        Forma cuadradoAzul = new Cuadrado(new Azul());  
  
        circuloRojo.dibujar();  
        cuadradoRojo.dibujar();  
        circuloAzul.dibujar();  
        cuadradoAzul.dibujar();  
  
    }  
}
```

ESTRUCTURAL - COMPOSITE

Es un patrón de diseño estructural que te permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.



El patrón Composite te proporciona dos tipos de elementos básicos que comparten una interfaz común: hojas simples y contenedores complejos.

Un contenedor puede estar compuesto por hojas y por otros contenedores. Esto te permite construir una estructura de objetos recursivos anidados parecida a un árbol.

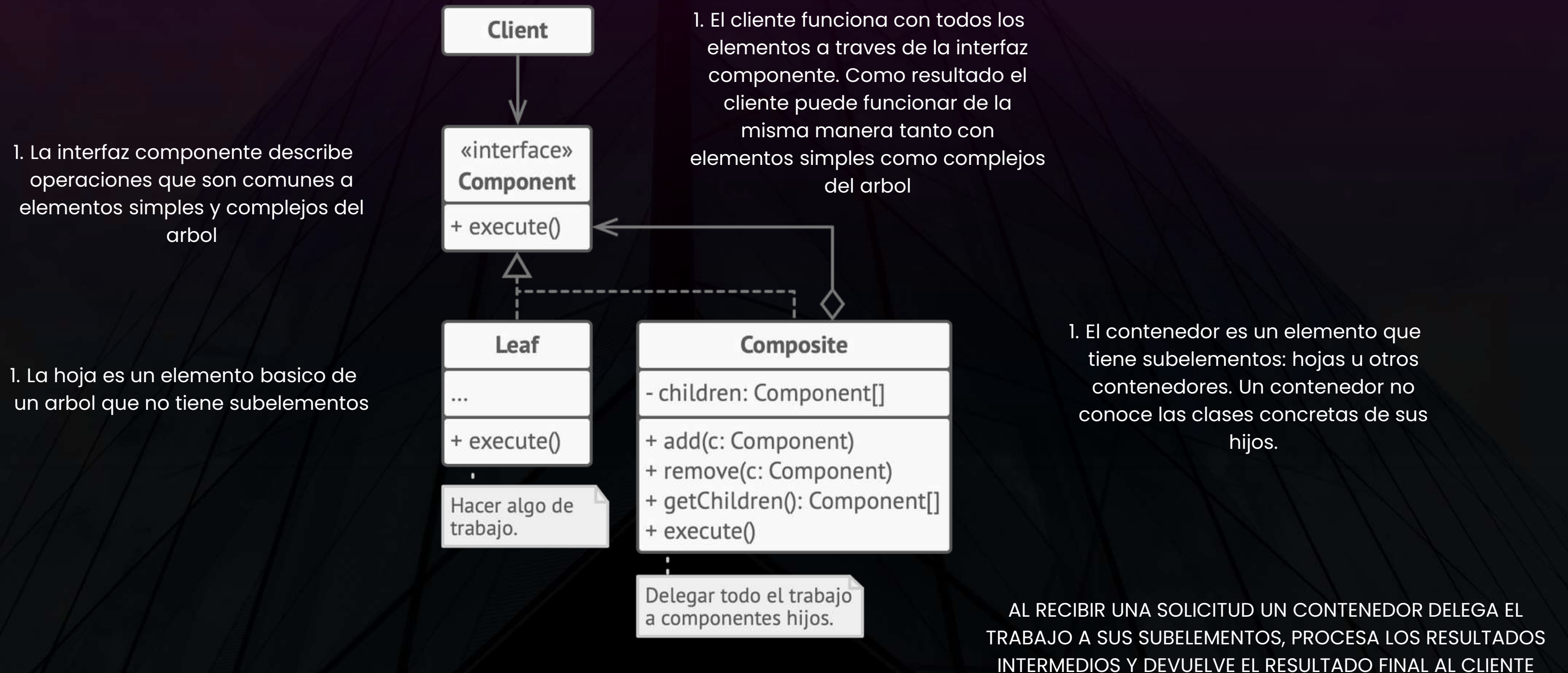


Puedes trabajar con estructuras de árbol complejas con mayor comodidad: utiliza el polimorfismo y la recursión en tu favor.



Todos los elementos definidos por el patrón Composite comparten una interfaz común. Utilizando esta interfaz, el cliente no tiene que preocuparse por la clase concreta de los objetos con los que funciona.

COMO IMPLEMENTARLO



EJERCICIO #6



Una bodega maneja productos individuales y cajas que pueden contener:

Productos

Otras cajas (cajas dentro de cajas)

El sistema debe poder calcular el precio total de cualquier elemento, sin importar si es un producto simple o una caja compuesta.

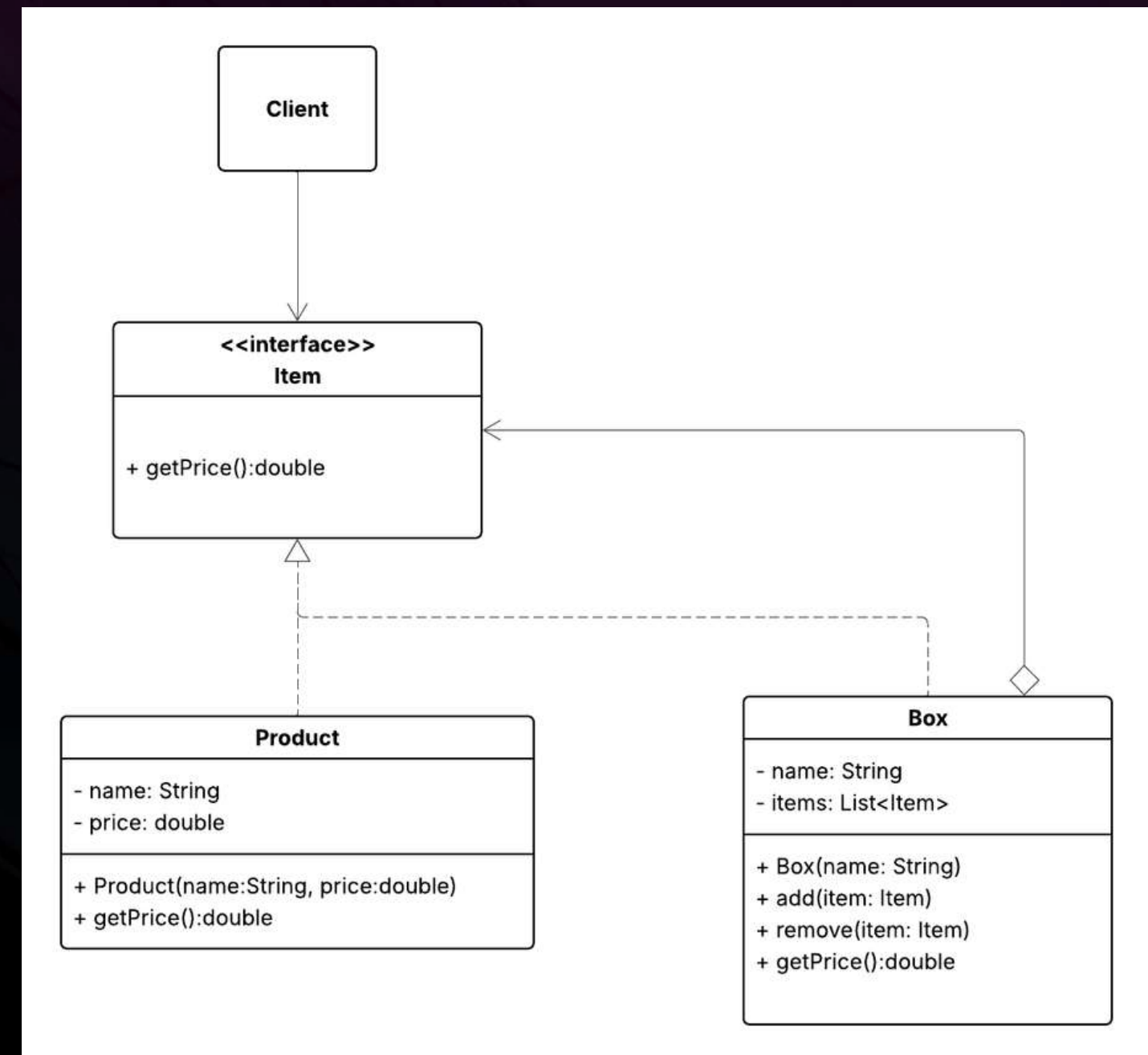
Para obtener el valor del precio total es necesario recorrer todos los items y obtener el precio

EJERCICIO #6



Una bodega maneja productos individuales y cajas que pueden contener:
Productos
Otras cajas (cajas dentro de cajas)
El sistema debe poder calcular el precio total de cualquier elemento, sin importar si es un producto simple o una caja compuesta.

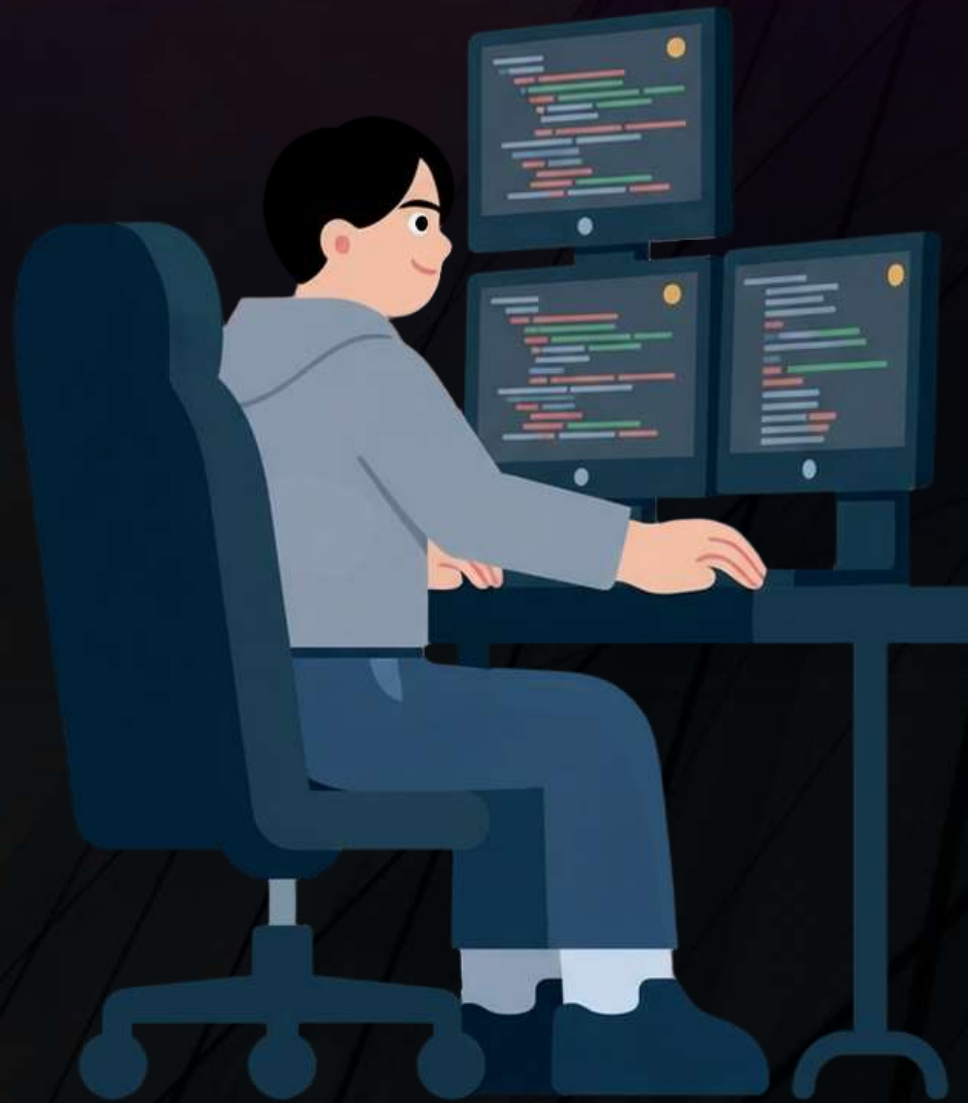
Para obtener el valor del precio total es necesario recorrer todos los items y obtener el precio





EJERCICIO #6

En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class WarehouseApp {  
  
    public static void main(String[] args) {  
  
        Product laptop = new Product("Laptop", 1200);  
        Product mouse = new Product("Mouse", 40);  
        Product keyboard = new Product("Keyboard", 80);  
  
        Box accessoriesBox = new Box("Accessories Box");  
        accessoriesBox.add(mouse);  
        accessoriesBox.add(keyboard);  
  
        Box mainBox = new Box("Main Box");  
        mainBox.add(laptop);  
        mainBox.add(accessoriesBox);  
  
        System.out.println("Total price: $" + mainBox.getPrice());  
    }  
}
```


ESTRUCTURAL - DECORATOR

Es un patrón de diseño estructural que te permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades.



✓ Cuando tenemos que alterar la funcionalidad de un objeto, lo primero que se viene a la mente es extender una clase. No obstante, la herencia tiene varias limitaciones importantes de las que debes ser consciente

✓ Vestir ropa es un ejemplo del uso de decoradores. Cuando tienes frío, te cubres con un suéter. Si sigues teniendo frío a pesar del suéter, puedes ponerte una chaqueta encima. Si está lloviendo, puedes ponerte un impermeable. Todas estas prendas “extienden” tu comportamiento básico pero no son parte de ti, y puedes quitarte fácilmente cualquier prenda cuando lo desees.

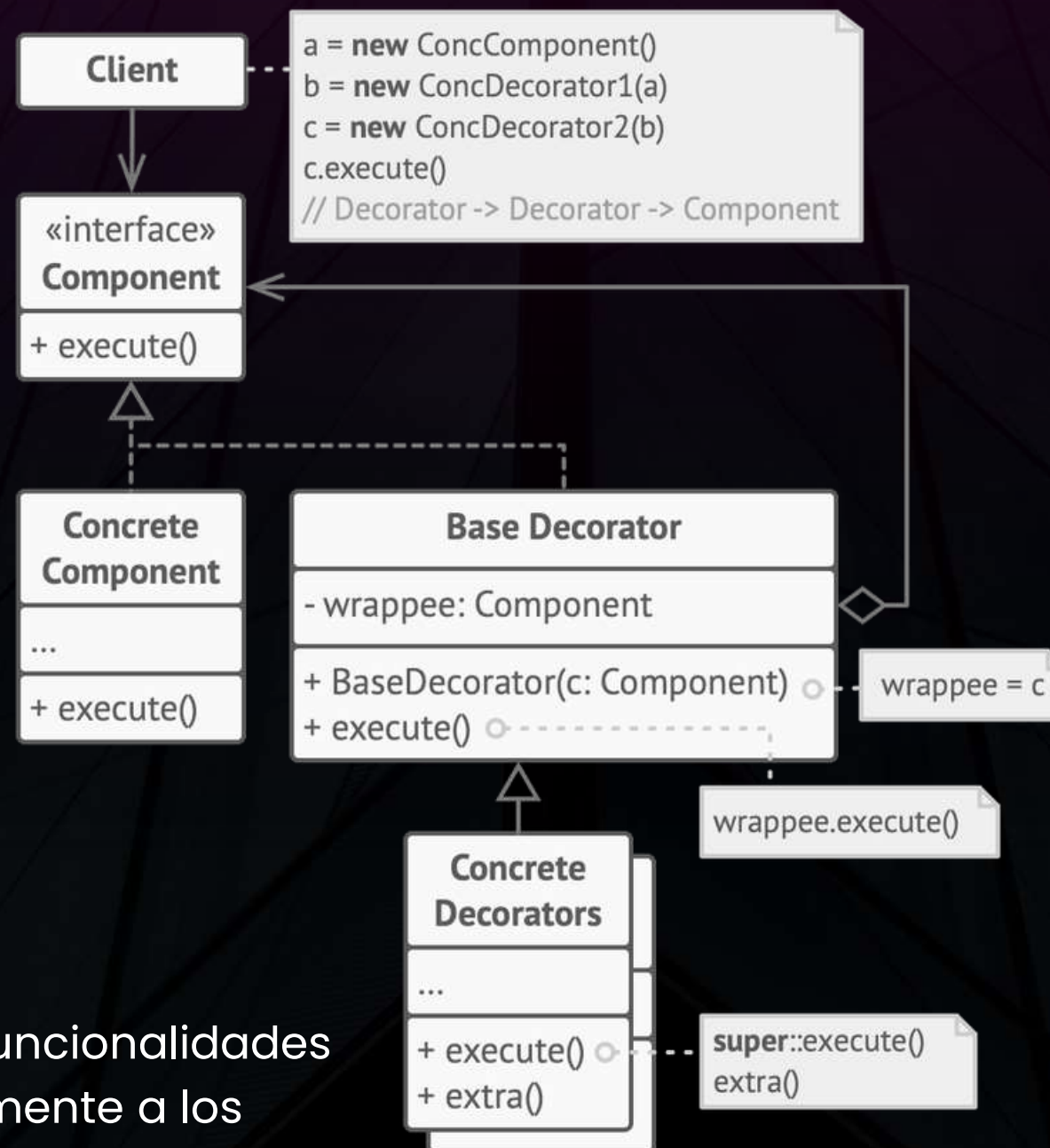
✓ Utiliza el patrón Decorator cuando necesites asignar funcionalidades adicionales a objetos durante el tiempo de ejecución sin descomponer el código que utiliza esos objetos.

COMO IMPLEMENTARLO

1. El Componente declara la interfaz común tanto para wrappers como para objetos envueltos.

1. El componente concreto es una clase de objetos envueltos, define el comportamiento basico que los decoradores pueden alterar

1. Los decoradores concretos definen funcionalidades que se pueden añadir dinamicamente a los componentes



1. La clase decoradora Base tiene un campo para referenciar un objeto envuelto, el tipo del campo debe declararse como la interfaz del componente para que pueda contener tanto los componentes concretos como los decoradores, la clase decoradora base delega todas las operaciones al objeto envuelto

EJERCICIO #7



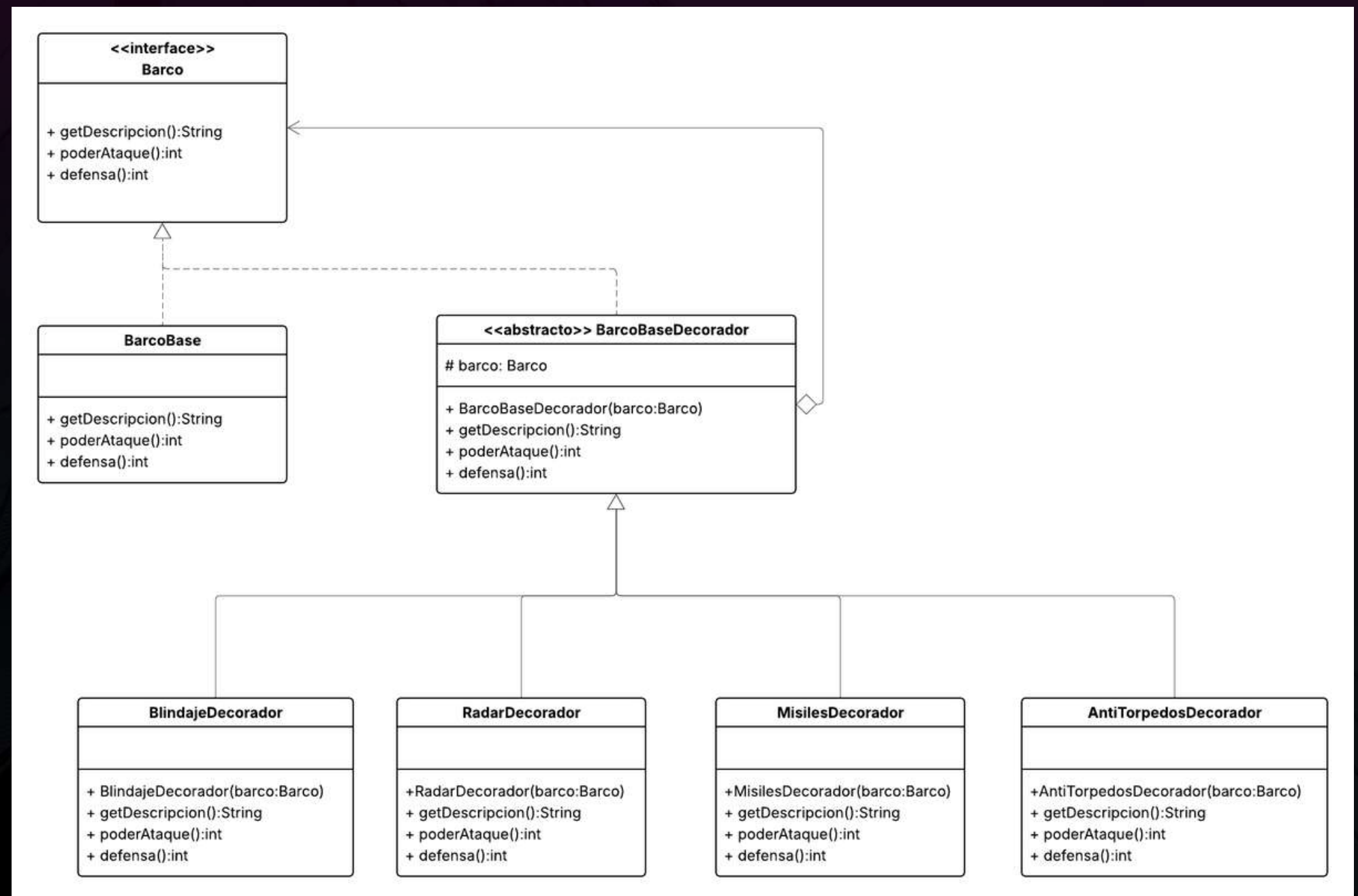
Estás desarrollando un simulador naval.

Cada barco tiene capacidades básicas como ataque, defensa y una descripción del tipo de barco, pero puede ser equipado con módulos adicionales sin modificar su clase original:

- Blindaje reforzado → + 30 en defensa
- Radar avanzado → +10 en ataque
- Misiles → +40 en ataque
- Sistema antitorpedos → +20 de ataque

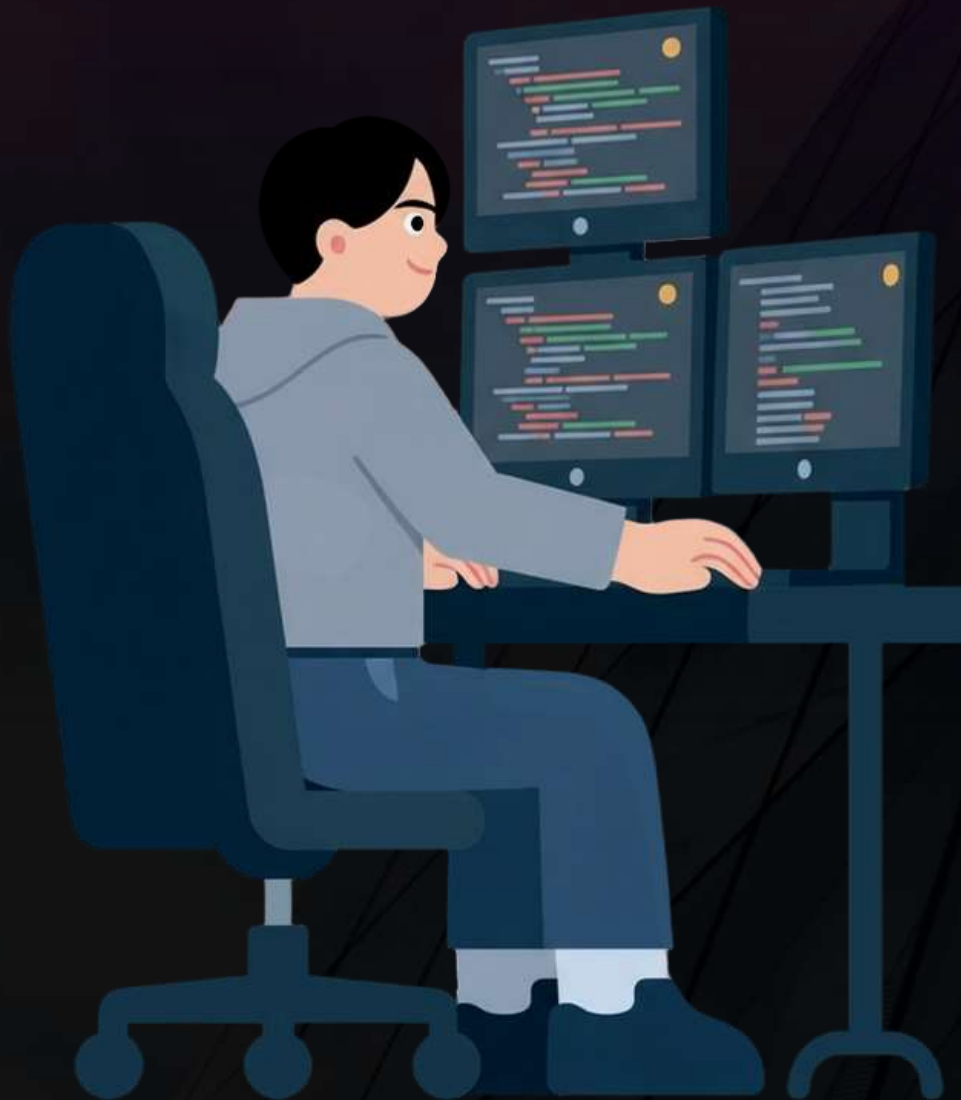
Un mismo barco puede tener múltiples mejoras

- combinar módulos dinámicamente
- cambiar su configuración en tiempo de ejecución
- No se permite crear subclases como BarcoConRadarYMisiles



EJERCICIO #7

En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class Main {  
  
    public static void main(String[] args) {  
  
        Barco barcoBase = new BarcoBase();  
  
        Map<String, Function<Barco, Barco>> mejoras = Map.of(  
            "BLINDAJE", BlindajeDecorator::new,  
            "RADAR", RadarDecorator::new,  
            "MISILES", MisilesDecorator::new,  
            "ANTITORPEDOS", AntitorpedosDecorator::new  
        );  
  
        List<String> configuracion = List.of(  
            "BLINDAJE",  
            "RADAR",  
            "MISILES"  
        );  
  
        Barco barcoFinal = configuracion.stream()  
            .map(mejoras::get)  
            .reduce(barcoBase,  
                (barco, decorador) -> decorador.apply(barco)  
            );  
    }  
}
```

```
List<String> configuracion = List.of(  
    "BLINDAJE",  
    "RADAR",  
    "MISILES"  
);  
  
Barco barcoFinal = configuracion.stream()  
    .map(mejoras::get)  
    .reduce(barcoBase,  
        (barco, decorador) -> decorador.apply(barco)  
    );  
  
System.out.println(barcoFinal.getDescripcion());  
System.out.println("Ataque: " + barcoFinal.poderAtaque());  
System.out.println("Defensa: " + barcoFinal.defensa());  
}
```


COMPORTAMIENTO - CHAIN OF RESPONSABILITY

Es un patrón de diseño de comportamiento que te permite pasar solicitudes a lo largo de una cadena de manejadores. Al recibir una solicitud, cada manejador decide si la procesa o si la pasa al siguiente manejador de la cadena.



El Chain of Responsibility se basa en transformar comportamientos particulares en objetos autónomos llamados manejadores. En nuestro caso, cada comprobación debe ponerse dentro de su propia clase con un único método que realice la comprobación. La solicitud, junto con su información, se pasa a este método como argumento.

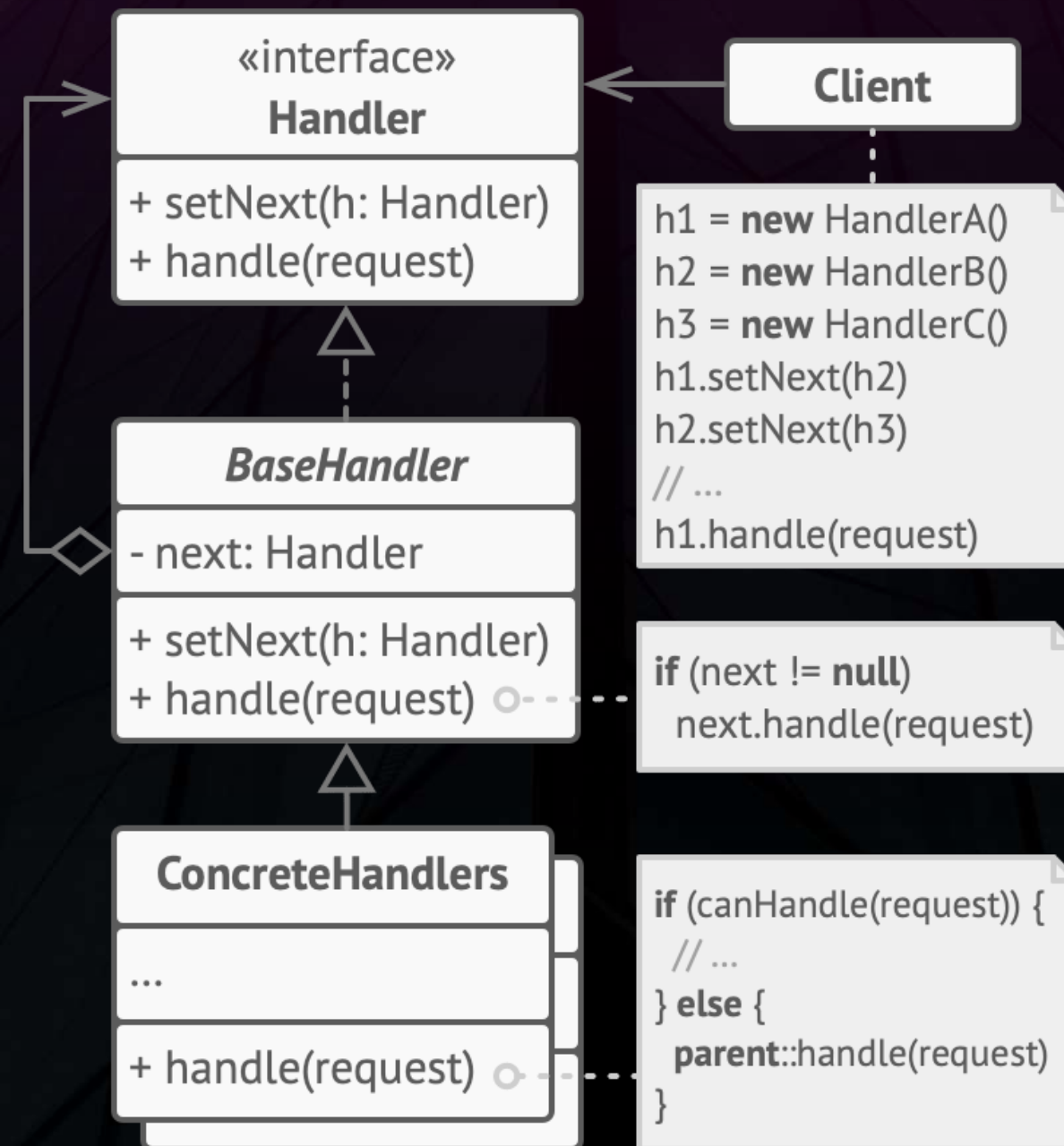
Utiliza el patrón Chain of Responsibility cuando tu programa deba procesar distintos tipos de solicitudes de varias maneras, pero los tipos exactos de solicitudes y sus secuencias no se conozcan de antemano.

Puedes controlar el orden de control de solicitudes.

COMO IMPLEMENTARLO

1. La clase manejadora declara la interfaz comun a todos los manejadores concretos

1. La clase manejadora base es opcional y es donde puedes colocar el codigo comun para todas las clases manejadoras, normalmente esta clase define un campo para almacenar una referencia al siguiente manejador



1. El cliente puede componer cadenas una sola vez o componerlas dinamicamente

1. Los manejadores concretos contienen el codigo para procesar las solicitudes. Al recibir una solicitud cada manejador debe decidir si procesarla y ademas si la pasa a lo largo de la cadena.

LOS MANEJADORES SON AUTONOMOS E INMUTABLES

EJERCICIO #8

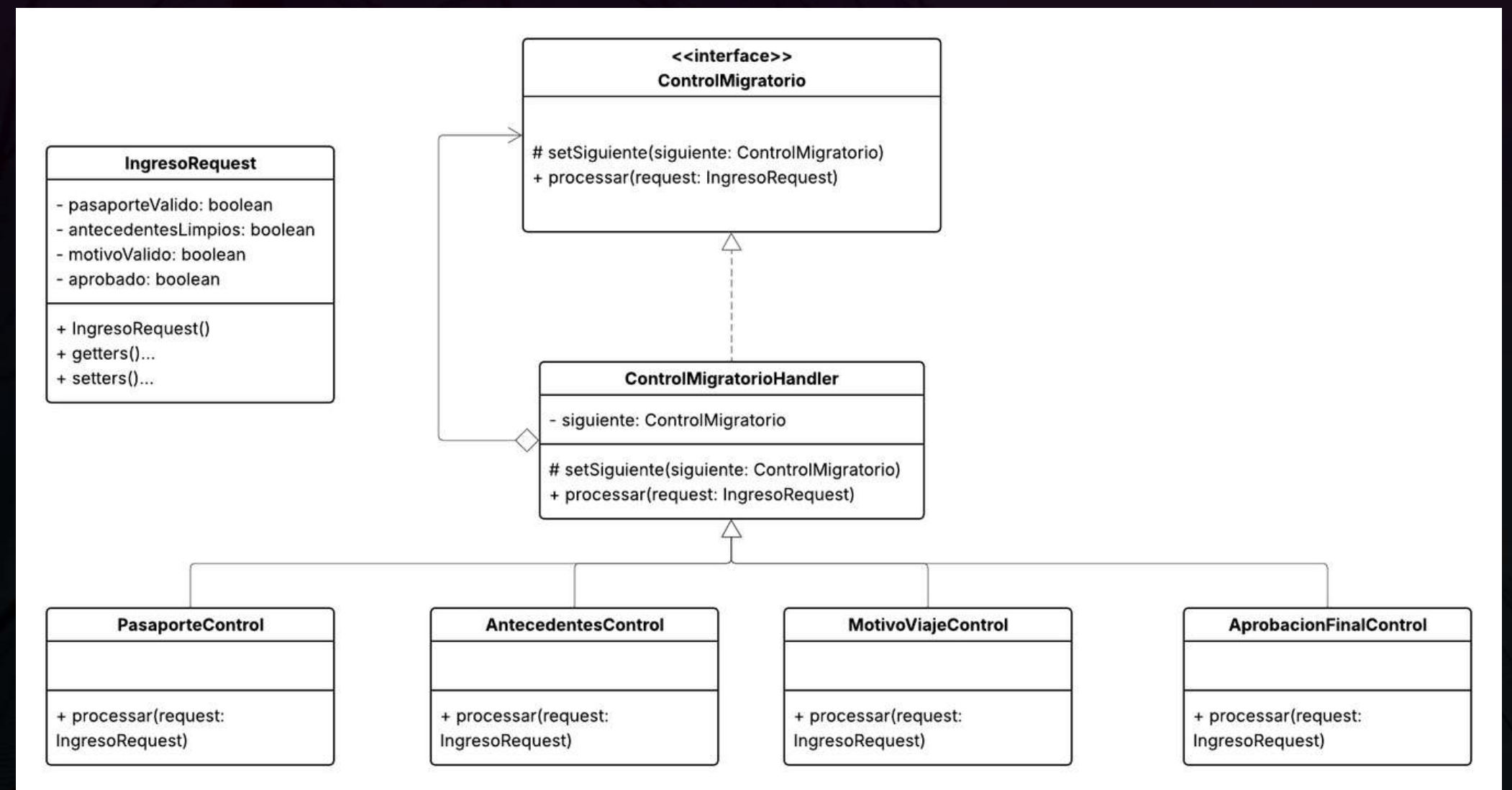


Una persona intenta ingresar a Estados Unidos.
Antes de permitir la entrada, debe pasar por una
cadena de controles migratorios:

- Revisión de pasaporte y visa
- Control de antecedentes
- Revisión de motivos del viaje
- Aprobación final por migración

Cada control:

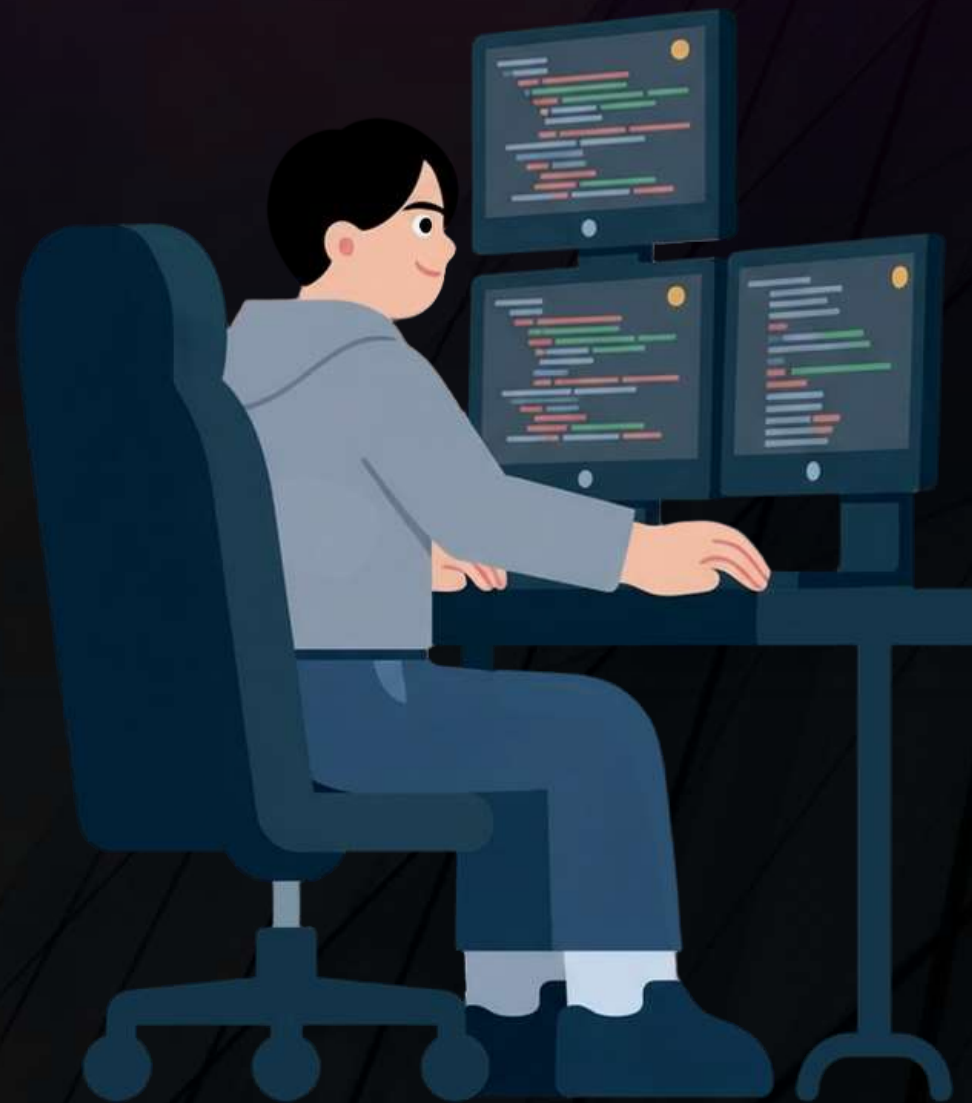
- Puede aprobar y pasar al siguiente
- O rechazar el ingreso y detener el proceso
- La persona no conoce qué control la rechaza ni cuántos controles existen.





EJERCICIO #8

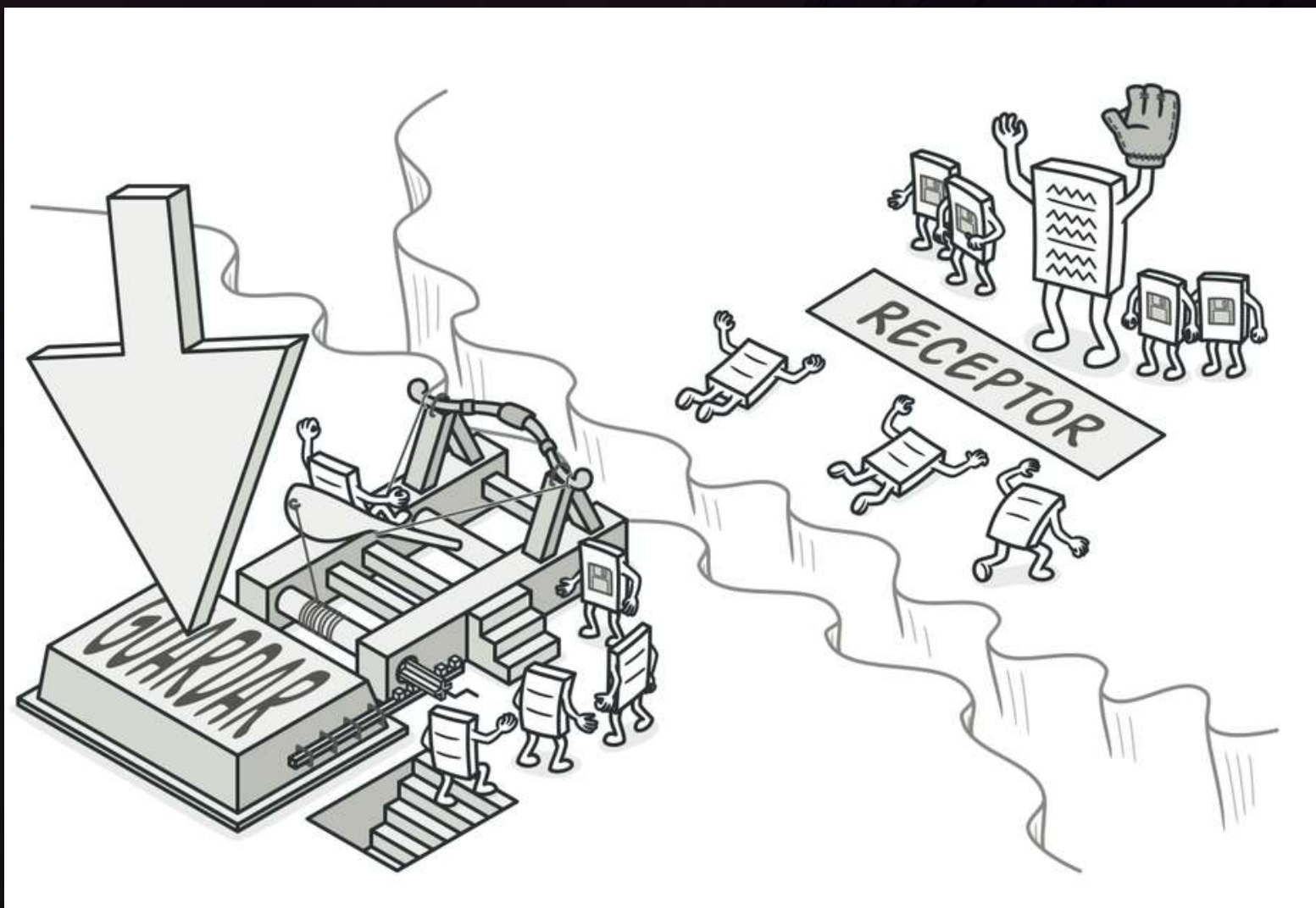
En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class Main {  
  
    public static void main(String[] args) {  
  
        ControlMigratorio pasaporte = new PasaporteControl();  
        ControlMigratorio antecedentes = new AntecedentesControl();  
        ControlMigratorio motivo = new MotivoViajeControl();  
        ControlMigratorio aprobacion = new AprobacionFinalControl();  
  
        pasaporte.setSiguiente(antecedentes);  
        antecedentes.setSiguiente(motivo);  
        motivo.setSiguiente(aprobacion);  
  
        IngresoRequest persona = new IngresoRequest(  
            true,  
            true,  
            false  
        );  
  
        pasaporte.procesar(persona);  
    }  
}
```


COMPORTAMIENTO - COMMAND

Es un patrón de diseño de comportamiento que convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.



✓ El patrón Command puede convertir una llamada a un método específico en un objeto autónomo. Este cambio abre la puerta a muchos usos interesantes: puedes pasar comandos como argumentos de método, almacenarlos dentro de otros objetos, cambiar comandos vinculados durante el tiempo de ejecución, etc.

✓ Utiliza el patrón Command cuando quieras poner operaciones en cola, programar su ejecución, o ejecutarlas de forma remota.

✓ Utiliza el patrón Command cuando quieras implementar operaciones reversibles.

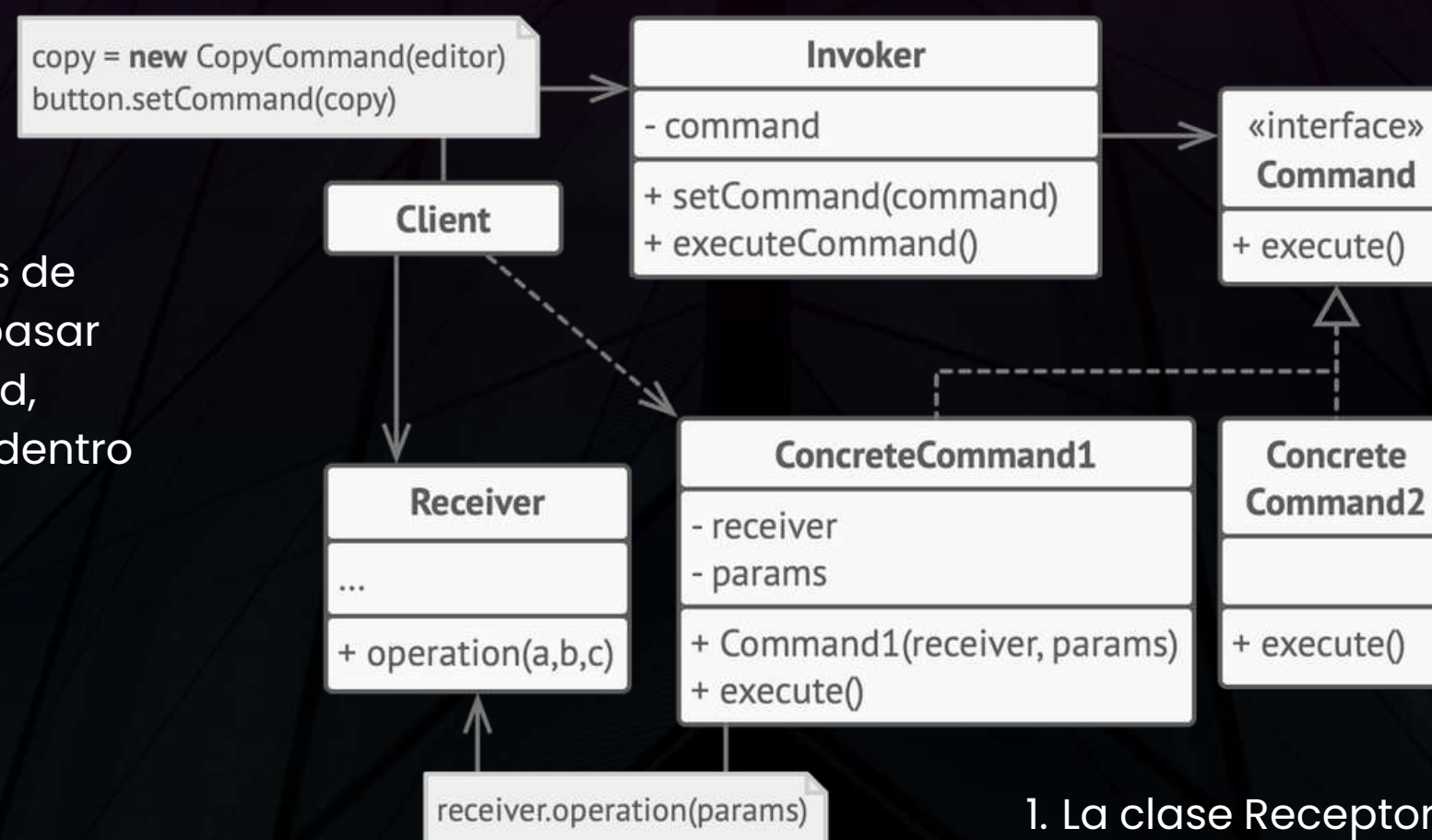
COMO IMPLEMENTARLO

1. La clase Emisora es responsable de inicializar las solicitudes. Esta clase debe tener un campo para almacenar una referencia a un objeto de comando. El emisor activa este comando en lugar de enviar la solicitud directamente al receptor

1. La interfaz comando normalmente declara un unico metodo para ejecutar el comando

1. El cliente crea y configura los objetos de comando concretos. El cliente debe pasar todos los parametros de la solicitud, incluyendo una instancia del receptor, dentro del constructor del comando

1. Los comandos concretos implementan varios tipos de solicitudes, un comando concreto no se supone que tenga que realizar el trabajo por su cuenta



1. La clase Receptora contiene cierta lógica de negocio. Casi cualquier objeto puede actuar como receptor. La mayoría de los comandos solo gestiona los detalles sobre cómo se pasa una solicitud al receptor, mientras que el propio receptor hace el trabajo real.

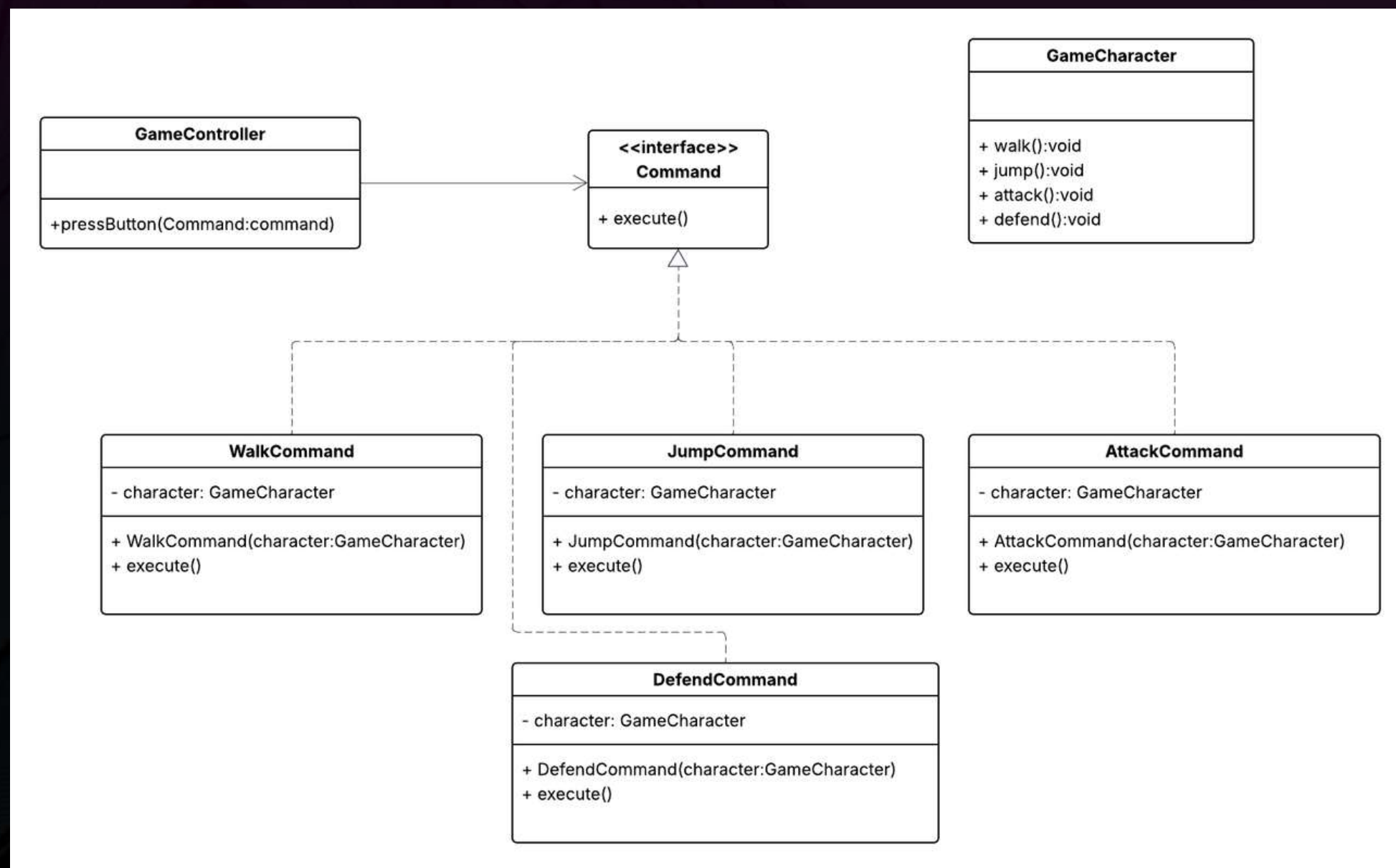
EJERCICIO #9



Un personaje de videojuego puede ejecutar diferentes acciones mediante comandos:

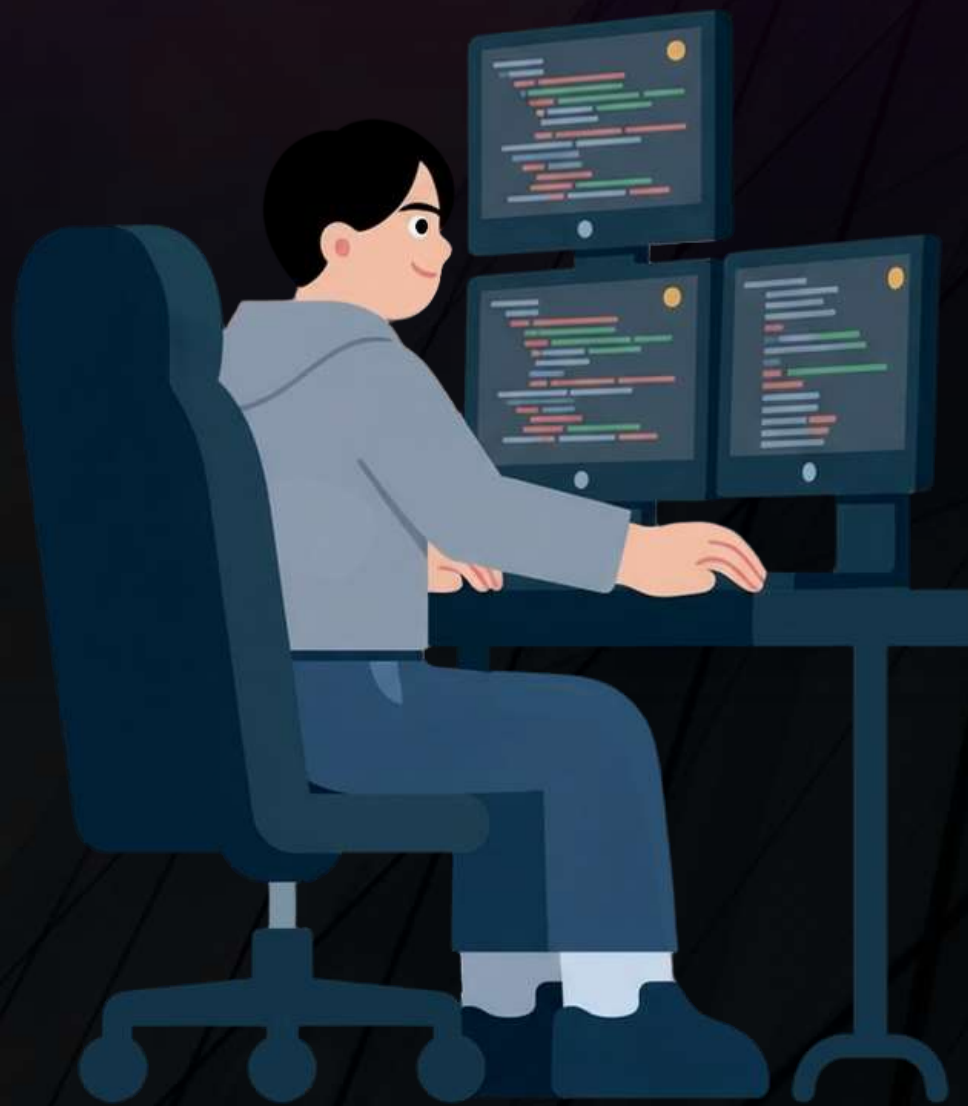
- Caminar
- Saltar
- Atacar
- Defenderse

Cada acción se encapsula como un comando, de modo que el control del juego pueda ejecutar cualquier acción sin conocer su implementación interna.



EJERCICIO #9

En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
import java.util.List;

public class Main {
    public static void main(String[] args) {

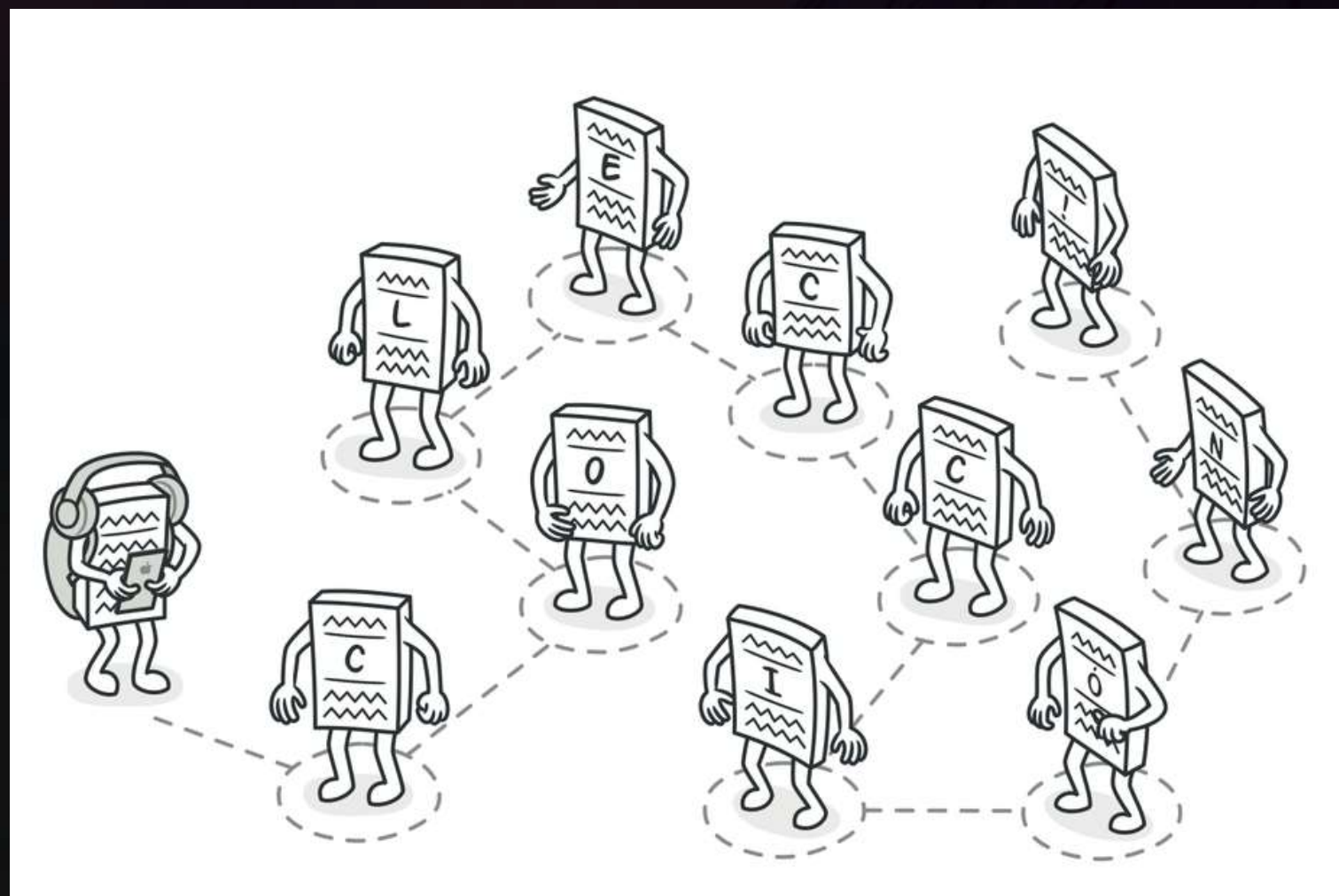
        GameCharacter character = new GameCharacter();
        GameController controller = new GameController();

        List<Command> actions = List.of(
            new WalkCommand(character),
            new JumpCommand(character),
            new AttackCommand(character),
            new DefendCommand(character)
        );

        actions.forEach(controller::pressButton);
    }
}
```


COMPORTAMIENTO - ITERATOR

Es un patrón de diseño de comportamiento que te permite recorrer elementos de una colección sin exponer su representación subyacente. Las colecciones son de los tipos de datos más utilizados en programación. Sin embargo, una colección tan solo es un contenedor para un grupo de objetos.



✓ La idea central del patrón Iterator es extraer el comportamiento de recorrido de una colección y colocarlo en un objeto independiente llamado iterador.

✓ Además de implementar el propio algoritmo, un objeto iterador encapsula todos los detalles del recorrido, como la posición actual y cuántos elementos quedan hasta el final. Debido a esto, varios iteradores pueden recorrer la misma colección al mismo tiempo, independientemente los unos de los otros.

✓ Utiliza el patrón para reducir la duplicación en el código de recorrido a lo largo de tu aplicación.

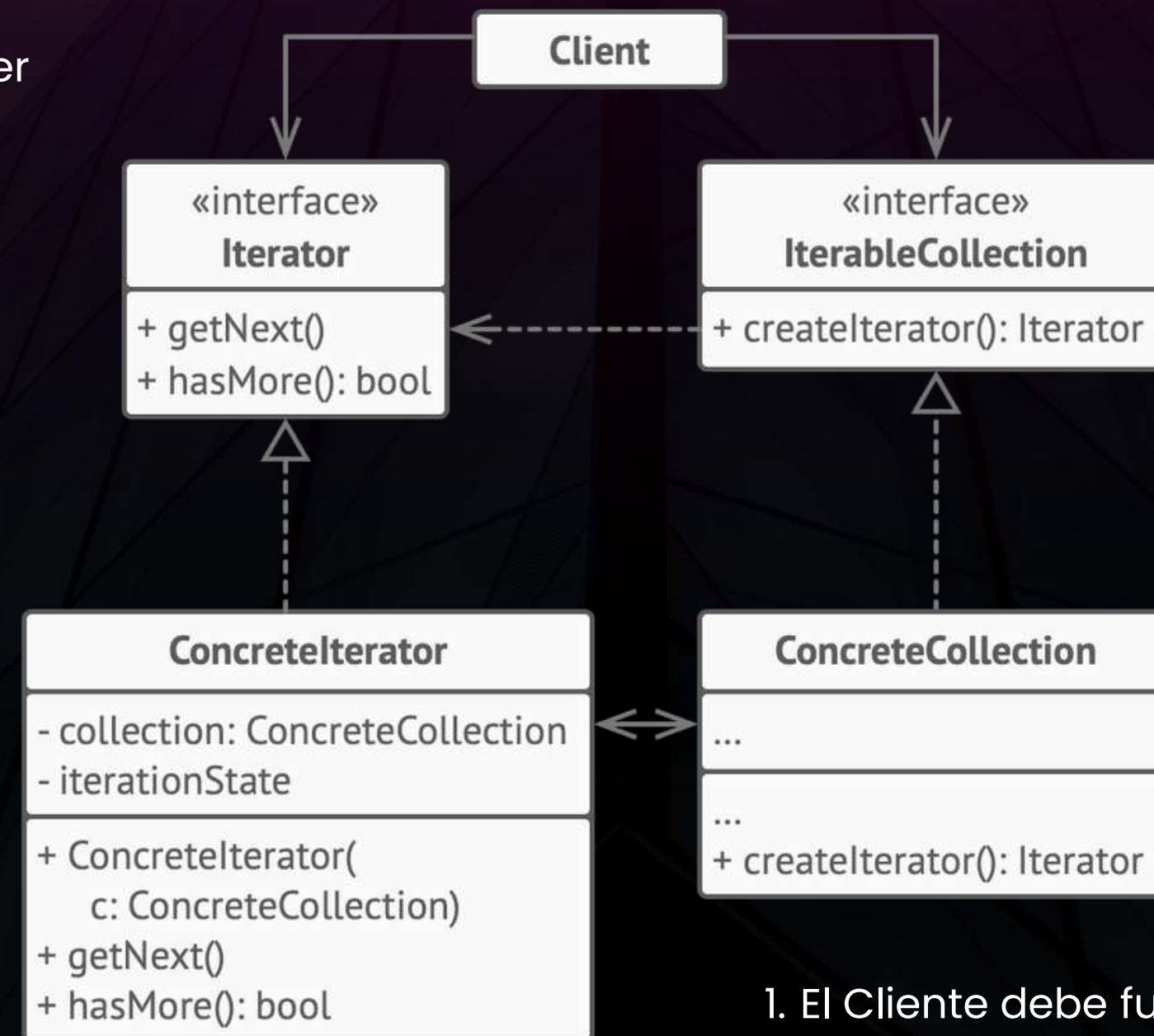
COMO IMPLEMENTARLO

1. La interfaz Iteradora declara las operaciones necesarias para recorrer una colección: extraer el siguiente elemento, recuperar la posición actual, reiniciar la iteración, etc.

1. Los Iteradores Concretos implementan algoritmos específicos para recorrer una colección. El objeto iterador debe controlar el progreso del recorrido por su cuenta.

1. La interfaz Colección declara uno o varios métodos para obtener iteradores compatibles con la colección.

1. Las Colecciones Concretas devuelven nuevas instancias de una clase iteradora concreta particular cada vez que el cliente solicita una.



1. El Cliente debe funcionar con colecciones e iteradores a través de sus interfaces. De este modo, el cliente no se acopla a clases concretas, permitiéndote utilizar varias colecciones e iteradores con el mismo código cliente.

EJERCICIO #10



Un turista quiere explorar distintos lugares emblemáticos de Roma:

- Coliseo
- Foro Romano
- Fontana di Trevi
- Panteón
- Plaza de España

El recorrido debe poder iterarse sin exponer la estructura interna que almacena los lugares.

EJERCICIO #10

1

```
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
}
```

```
public interface Aggregate<T> {  
    Iterator<T> createIterator();  
}
```

Esto representa cualquier coleccion que pueda producir un iterador

2

```
public class Place {  
    private final String name;  
  
    public Place(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```




DOSW COMPANY

EJERCICIO #10

3

```
public class TourRoute implements Aggregate<Place> {  
  
    private final Place[] places;  
  
    public TourRoute() {  
        places = new Place[] {  
            new Place("Colosseum"),  
            new Place("Roman Forum"),  
            new Place("Trevi Fountain"),  
            new Place("Pantheon"),  
            new Place("Spanish Steps")  
        };  
    }  
  
    public Iterator<Place> createIterator() {  
        return new RomeIterator();  
    }  
}
```

```
private class RomeIterator implements Iterator<Place> {  
  
    private int currentIndex = 0;  
  
    public boolean hasNext() {  
        return currentIndex < places.length;  
    }  
  
    public Place next() {  
        return places[currentIndex++];  
    }  
}
```

EJERCICIO #10

4

```
public class Tourist {  
  
    public void exploreTour(Aggregate<Place> route) {  
        Iterator<Place> iterator = route.createIterator();  
  
        while (iterator.hasNext()) {  
            Place p = iterator.next();  
            System.out.println("Visiting " + p.getName());  
        }  
    }  
}
```

5

```
public class Main {  
  
    public static void main(String[] args) {  
  
        TourRoute roma = new TourRoute();  
        Tourist tourist = new Tourist();  
  
        tourist.exploreTour(roma);  
    }  
}
```


COMPORTAMIENTO - STRATEGY

Es un patrón de diseño de comportamiento que te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.



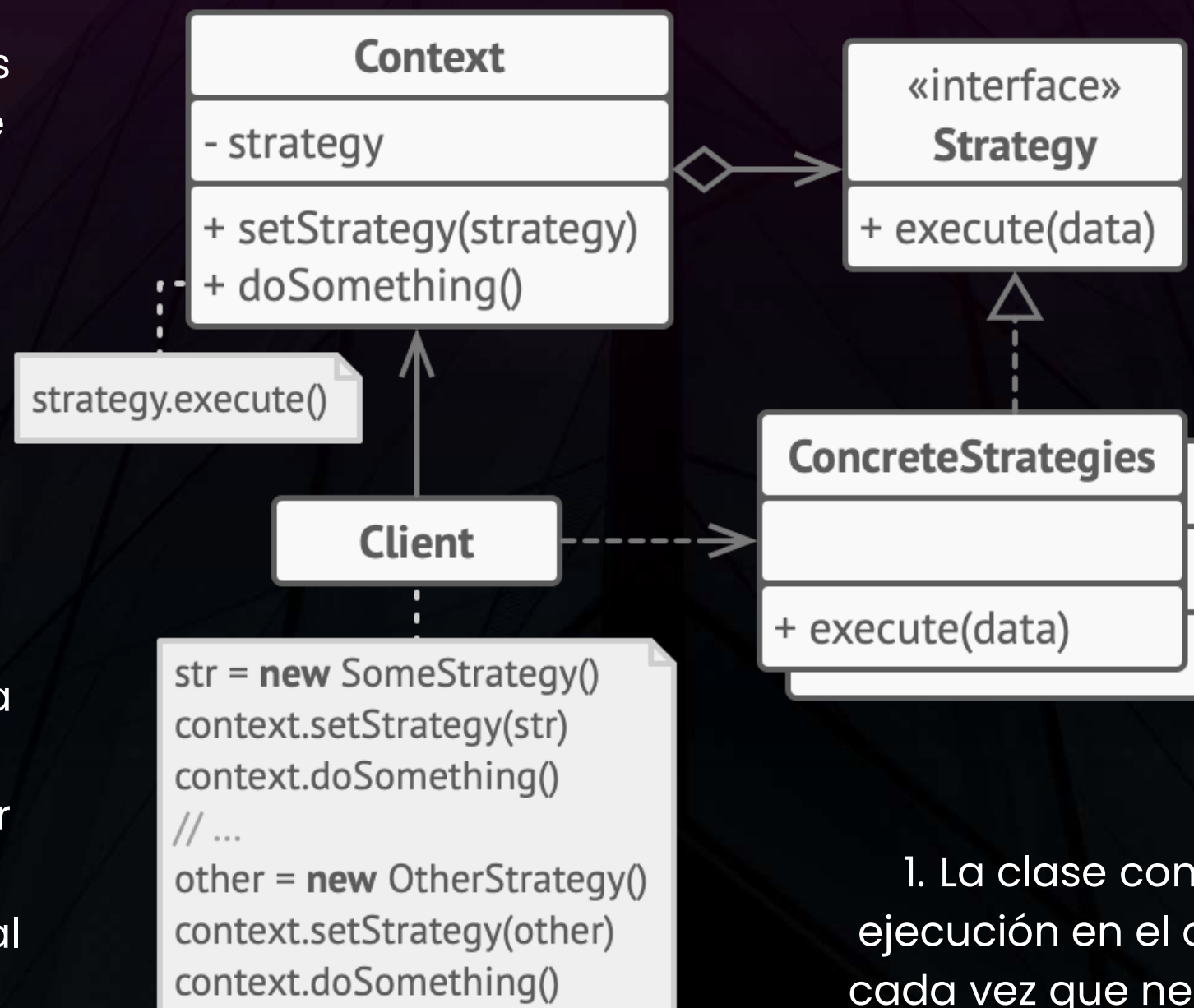
✓ El patrón Strategy sugiere que tomes esa clase que hace algo específico de muchas formas diferentes y extraigas todos esos algoritmos para colocarlos en clases separadas llamadas estrategias.

✓ Utiliza el patrón Strategy cuando quieras utilizar distintas variantes de un algoritmo dentro de un objeto y poder cambiar de un algoritmo a otro durante el tiempo de ejecución.

✓ Utiliza el patrón Strategy cuando tengas muchas clases similares que sólo se diferencien en la forma en que ejecutan cierto comportamiento.

COMO IMPLEMENTARLO

1. La clase Contexto mantiene una referencia a una de las estrategias concretas y se comunica con este objeto únicamente a través de la interfaz estrategia.



1. La interfaz Estrategia es común a todas las estrategias concretas. Declara un método que la clase contexto utiliza para ejecutar una estrategia.

1. Las Estrategias Concretas implementan distintas variaciones de un algoritmo que la clase contexto utiliza.

1. El Cliente crea un objeto de estrategia específico y lo pasa a la clase contexto. La clase contexto expone un modificador set que permite a los clientes sustituir la estrategia asociada al contexto durante el tiempo de ejecución.

1. La clase contexto invoca el método de ejecución en el objeto de estrategia vinculado cada vez que necesita ejecutar el algoritmo. La clase contexto no sabe con qué tipo de estrategia funciona o cómo se ejecuta el algoritmo.

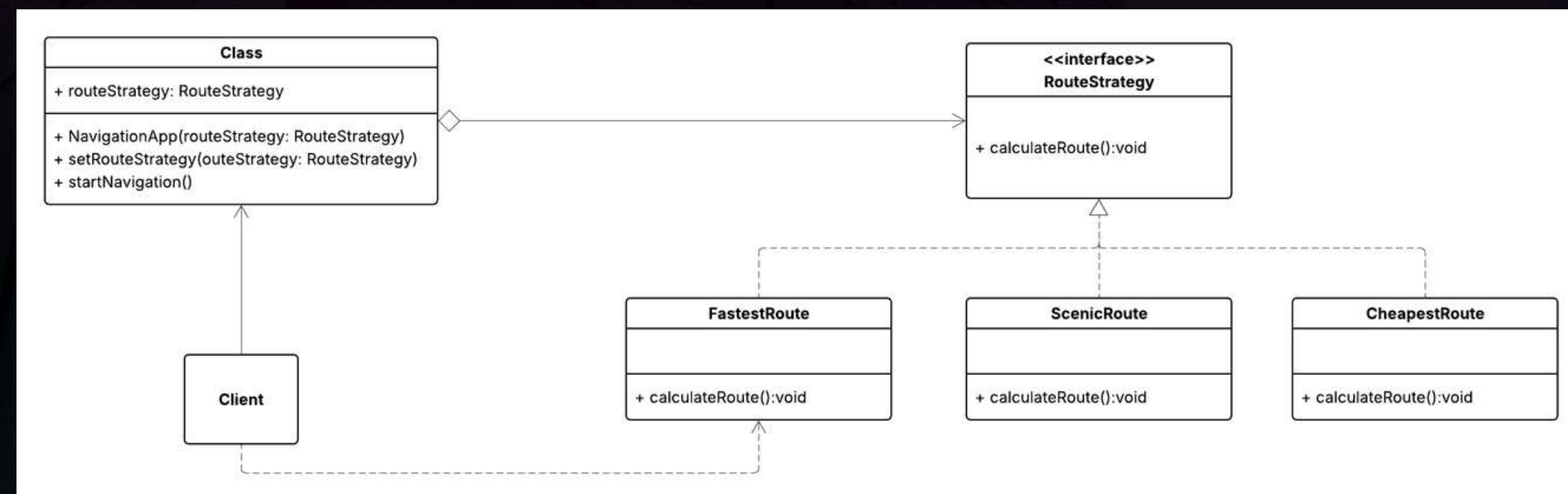
EJERCICIO #11



Una aplicación de navegación puede calcular rutas de distintas maneras.

El algoritmo de cálculo de ruta no debe estar acoplado a la aplicación, ya que puede cambiar según la preferencia del usuario.

Permitir cambiar cómo se calcula la ruta sin modificar la app.



EJERCICIO #11

En código deben ejecutarlo desde una clase de la siguiente manera para validar su solución



```
public class Main {  
    public static void main(String[] args) {  
  
        NavigationApp app = new NavigationApp(new FastestRoute());  
        app.startNavigation();  
  
        app.setRouteStrategy(new ScenicRoute());  
        app.startNavigation();  
  
        app.setRouteStrategy(new CheapestRoute());  
        app.startNavigation();  
    }  
}
```


WE'RE GOING TO BOOTCAMP





LABORATORIO #2

Se les compartió el enunciado por el material de Teams, sus grupos de trabajo son aleatorios, podrán revisarlo y pedir el contacto de sus compañeros



NOS VEMOS
EN LA
PROXIMA SESION DE
LABORATORIO

